

Recitation 9

Neural Networks

Agenda

1. Basics
2. Layers
3. Classification
 - Methods
 - Other tasks

Learning

Neural Networks

PyTorch

Classification

The Goal of Learning (Traditional)

- We wish to learn a mapping $f(x)$ from a domain set $\mathcal{X}$ and a label set $\mathcal{Y}$
- Two common settings:
 - **Classification:** $\mathcal{Y}$ is discrete, where each $y \in \mathcal{Y}$ represents a “class”
 - **Regression:** $\mathcal{Y}$ is continuous

x	y	$f(x)$
Images of animal	{cat, dog, deer, ...}	Recognize the animal in the image
Sounds of speech	Words in English	Converts speech to text

The Goal of Learning (Traditional)

- We wish to learn a mapping $f(x)$ from a domain set $\mathcal{X}$ and a label set $\mathcal{Y}$
- Two common settings:
 - **Classification:** $\mathcal{Y}$ is discrete, where each $y \in \mathcal{Y}$ represents a “class”
 - **Regression:** $\mathcal{Y}$ is continuous

$\mathcal{X}$	$\mathcal{Y}$	$f(x)$
Images of animal	{cat, dog, deer, ...}	Recognize the animal in the image
Sounds of speech	Words in English	Converts speech to text
Images of faces	$\mathbb{R}^2$	Returns location of the mouth in the image
Corrupted images	Natural images	Restores a corrupted image to its original version

Learning Parametric Functions

Training set: collection of “labeled samples” $S = \{(x_i \in \mathcal{X}, y_i \in \mathcal{Y})\}_{i=1}^N$

Hypothesis space: a set of parametric functions $\mathcal{H} = \{f_\theta(x) | \theta \in \mathbb{R}^s\}$

- Examples:
 - Linear functions $f(x) = \sum_{i=1}^s x_i \cdot w_i$, parameterized by vector $w \in \mathbb{R}^s$
 - Linear transformations $f(x) = Ax$, parameterized by matrix $A \in \mathbb{R}^{d \times s}$
 - Neural networks

Goal: find parameters θ s.t. $f_\theta \in \mathcal{H}$ “best fit” the training set S

Not all parameters are learned:

- Learned parameters
- Hyperparameters – not learned.

What does it mean “best fit”?

Loss function: $L(f, S)$ measure the “error” of f w.r.t. S

- Lower loss means better fit

Goal: find f^* that minimizes the loss: $f^* = \operatorname{argmin}_{f \in \mathcal{H}} L(f, S)$

Examples:

- **“0-1 Loss”**: the percentage of examples on which f is wrong

$$L_{0-1}(f, S) = \frac{\text{number of times } f(x_i) \neq y_i}{|S|}$$

- **“Mean Square Error”**: average distance of $f(x_i)$ from y_i

$$L_{MSE}(f, S) = \frac{1}{|S|} \sum_i \|f(x_i) - y_i\|^2$$

- **“Cross Entropy Loss”**: Assume $f(x)$ outputs the probabilities of x belonging to each possible class. Use probability of choosing the correct class on all examples.

$$Loss = -\frac{1}{N} \sum_{j=1}^N \log(f(x_{j,correct}))$$

- where $f(x_{correct})$ is the probability assigned by the model to the correct class for x .

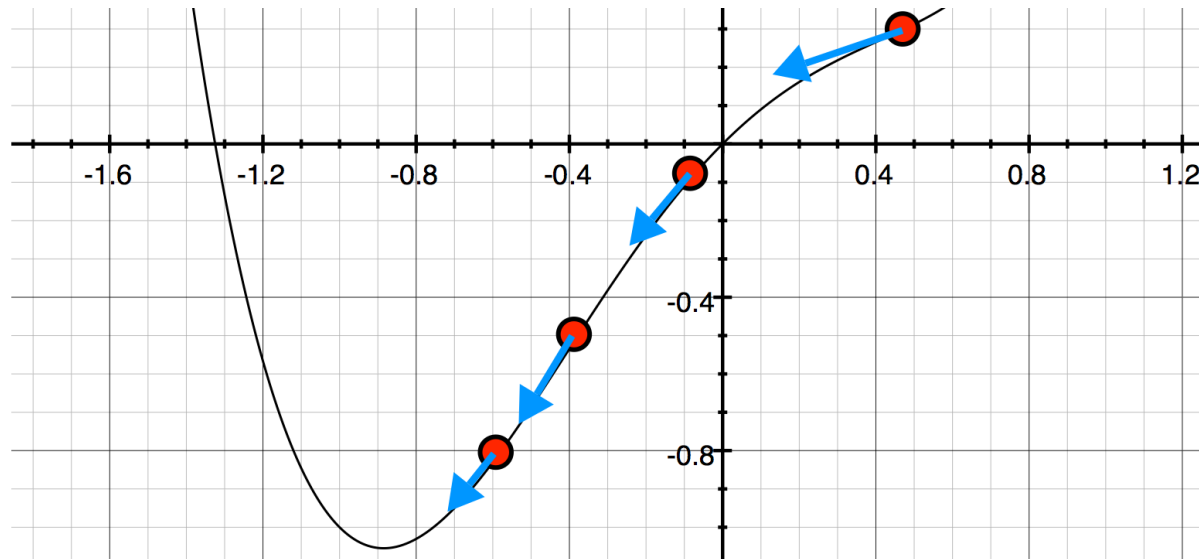
Minimizing the Loss Function

If $L(f, S)$ is a differentiable function, then we can use **Gradient Descent**

Key principle: at each iteration i of GD we take a step in the direction of steepest descent.

$$\theta_i = \theta_{i-1} - \eta \nabla f_{\theta_{i-1}}$$

Step size η (learning rate) is implementation specific (hyperparameter)



Minimizing the Loss Function

if $|S|$ is large, then every GD iteration is expensive

Instead, we approximate it using a small randomly chosen batch $B \subset S$:

Stochastic Gradient Descent

SGD loop:

1. Sample a random batch $B \subset S$
2. Compute the gradient of $L(f, B)$ w.r.t. the parameters of f
3. Update parameters of f using the gradient

We perform enough steps until we have covered the entire dataset – called “**epoch**”

We perform multiple epochs during training (i.e. multiple passes on entire dataset)

Original SGD could be difficult to use for non-experts

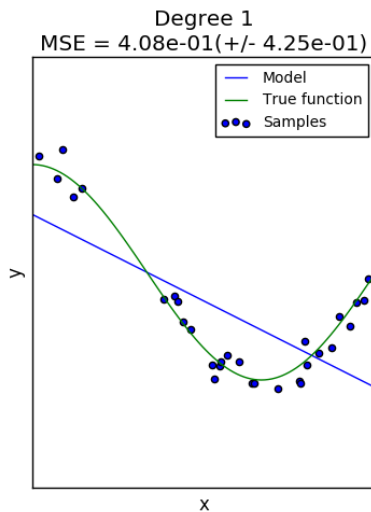
- ADAM is an SGD variant which is easier to use in practice

Generalization and Overfitting

Assume we have a model that fits the training set

Q: is it really good?

- A: not necessarily, it could **overfit** the data



We must evaluate the model on unseen data:

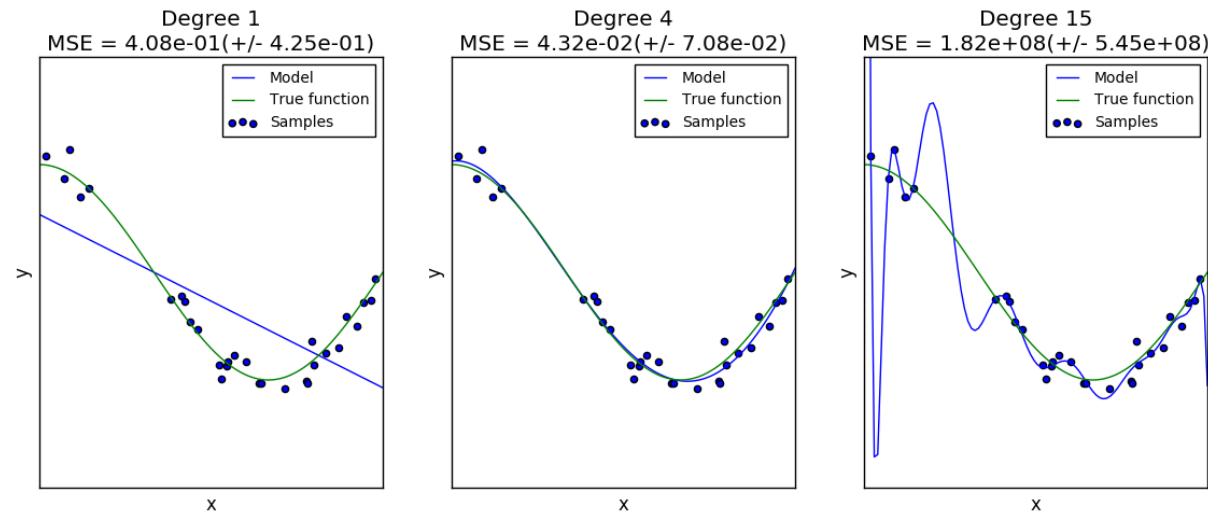
- Before training: split data into training, validation and test sets
- During training: choose hyperparameters using the validation set
- After training: evaluate final model on test set (generalization error)

Generalization and Overfitting

Assume we have a model that fits the training set

Q: is it really good?

- A: not necessarily, it could **overfit** the data



We must evaluate the model on unseen data:

- Before training: split data into training, validation and test sets
- During training: choose hyperparameters using the validation set
- After training: evaluate final model on test set (generalization error)

Learning Summary

We wish to fit a parametric function $f_{\theta}(x)$ to a training set S

We measure the fitness with a loss function $L(f_{\theta}, S)$:

- For classification we typically use “Cross-Entropy Loss”
- For regression we typically use “Mean Square Error”

We find the best f_{θ} by minimizing the loss

For minimization we use SGD or one of its variants

Beware of overfit! Always evaluate your model on unseen data

Learning

Neural Networks

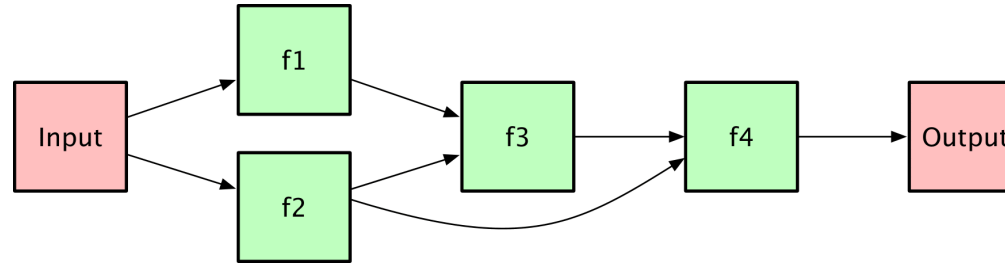
PyTorch
Classification



Video Source: Clip from *Silicon Valley*, HBO. © HBO.

Neural Network Definition

In broad terms: A directed acyclic graph of differentiable operations



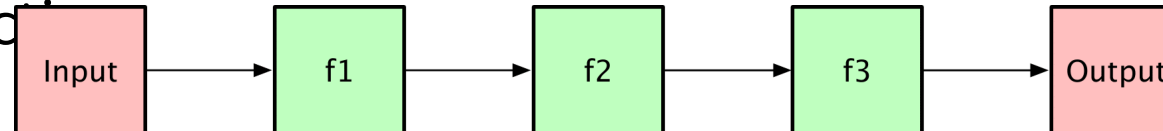
Each node is called a “**layer**”, defined by its type and parameters

All the parameters together are called the “**weights**” of the network

The “**depth**” of the network is the longest path from input to output

When learning we fix the structure (“**architecture**”) and only learn the weights

A network is called “**feed-forward**” if every node has at most one input and one output connected



Intermediate Results as “Tensors”

Tensors are a fancy term for a multi-dimensional array $A \in \mathbb{R}^{M_1 \times \dots \times M_n}$

- A is a nD tensor with the shape being the tuple $(M_1, \dots, M_n)$

Images are 3D tensors of the shape $(channels, height, width)$

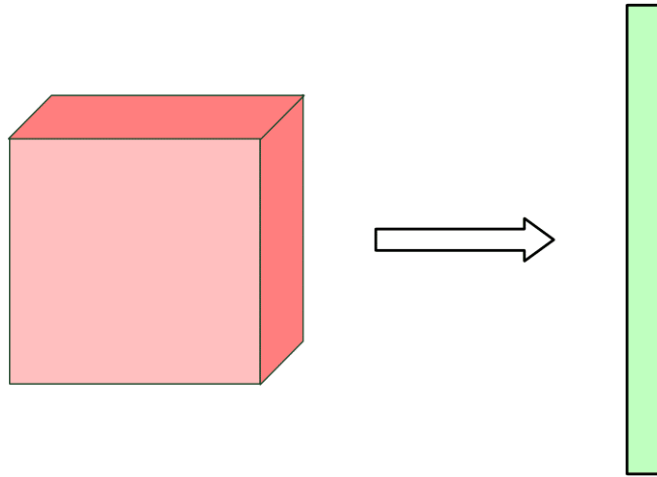
- For RGB: $channels = 3$. For grayscale: $channels = 1$.
- The convention in PyTorch is NCHW. Some other frameworks use NHWC (Keras for example)

The input/output of each layer are expressed as tensors

The shape and dimension of the output depends on the type of the layer

Common Layers: Linear / Fully-Connected

Operation: $f(x) = Ax + b$



Input: assume $x \in \mathbb{R}^N$.

Output: $y \in \mathbb{R}^M$

Hyperparameters: M – output dimension

Learned parameters: matrix $A \in \mathbb{R}^{N \times M}$ (“weights”) and vector $b \in \mathbb{R}^M$ (“bias”)

Common Layers: Activation

Operation: Apply point-wise non-linear function $\sigma: \mathbb{R} \rightarrow \mathbb{R}$, called “activation function”, on every entry (neuron) of the input tensor.

- **ReLU:** $\sigma(z) = \max(z, 0)$
- **Hyperbolic Tangent:** $\sigma(z) = \tanh(z)$
- **Sigmoid:** $\sigma(z) = \frac{1}{1 + \exp(-z)}$

Input: any tensor of some shape

Output: same shape as input

Hyperparameters: typically, none

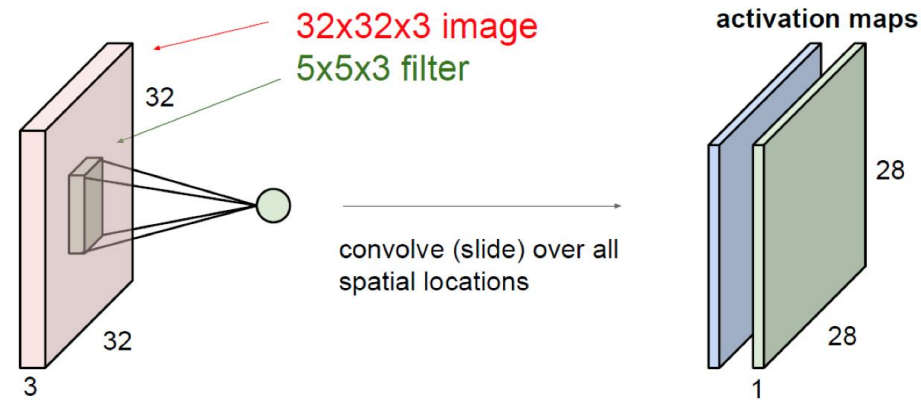
- Exception - LeakyReLU: $\sigma(z) = \max(z, 0.01z)$

Learned parameters: typically, none

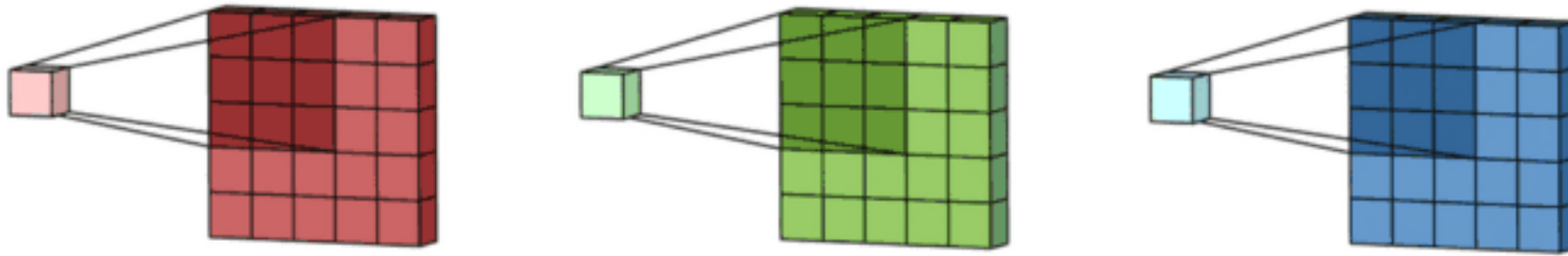
- Exception – PReLU: $\sigma(z) = \max(z, \alpha z)$ where α is learned

Common Layers: Convolution

Operation: convolve the input with a set of kernels



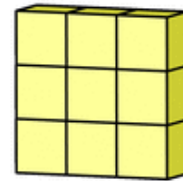
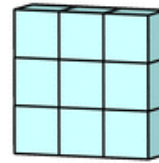
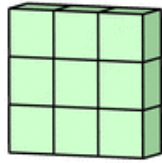
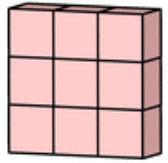
Common Layers: Convolution



Animation from the blog post: **“Intuitively Understanding Convolutions for Deep Learning”**

<https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

Common Layers: Convolution



Animation from the blog post: **“Intuitively Understanding Convolutions for Deep Learning”**

<https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

Common Layers: Convolution

Operation: convolve the input with a set of kernels

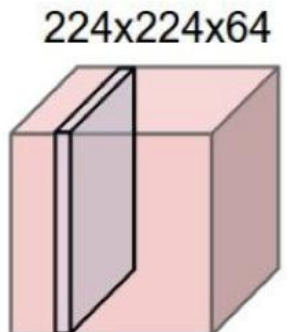
Input: a 3D tensor of shape $(in_channels, height, width)$

Output: a 3D tensor of shape $(out_channels, height, width)$

Hyperparameters:

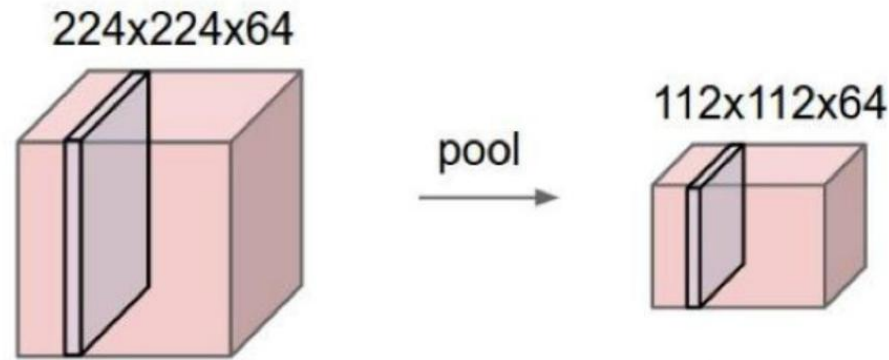
1. Spatial shape of kernel $K \times K$
2. Number of kernels M (number of output channels)

Learned parameters: for $1 \leq j \leq M$ the kernel $w_j \in \mathbb{R}^{C \times K \times K}$, where C is the number of input channels



Common Layers: Pooling

Operation: sub-sample each input channel



- **Average Pooling:** replace every window with its average value
- **Max Pooling:** replace every window with its maximum value

Input: a 3D tensor of shape $(in_channels, height, width)$

Output: a 3D tensor of shape $(in_channels, \frac{height}{window-height}, \frac{width}{window-width})$

Hyperparameters: size of pooling window (typically 2×2)

Learned parameters: None

Learning
Neural Networks

PyTorch

Classification

PyTorch

There are many deep learning frameworks: keras, tensorflow, PyTorch etc.

- We will be using PyTorch

API is very similar to numpy

Data and models are loaded by default to CPU

- Neural network computations are very computationally expensive and very slow on CPU
- We prefer running on GPU, where the computations are much faster
- In PyTorch we have to explicitly move tensors and the model to the GPU (and back to CPU)

Datasets and Neural networks are written as classes

See:

- <https://pytorch.org/tutorials/beginner/basics/intro.html>
- https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html

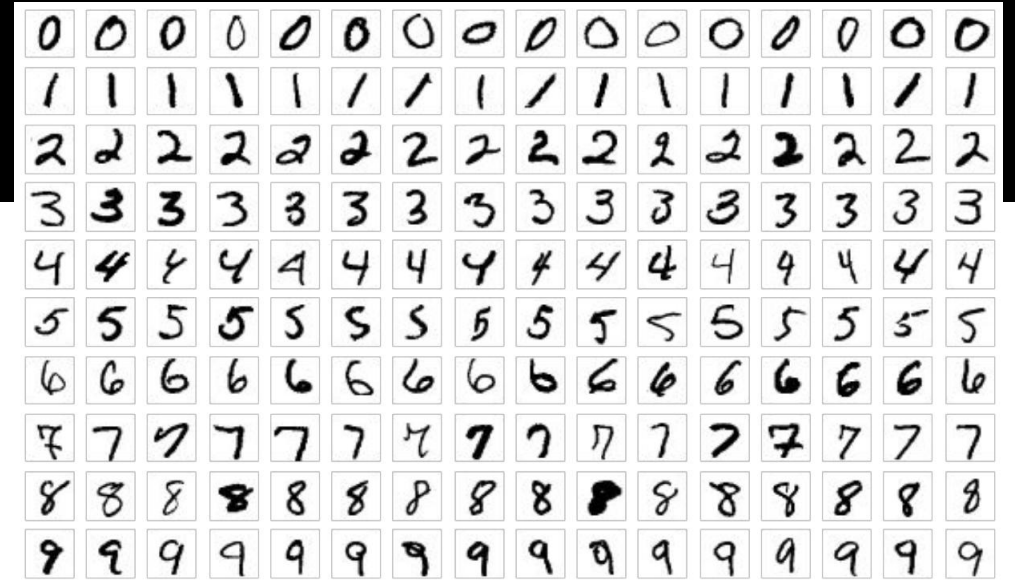
Learning
Neural Networks
PyTorch

Classification

Datasets

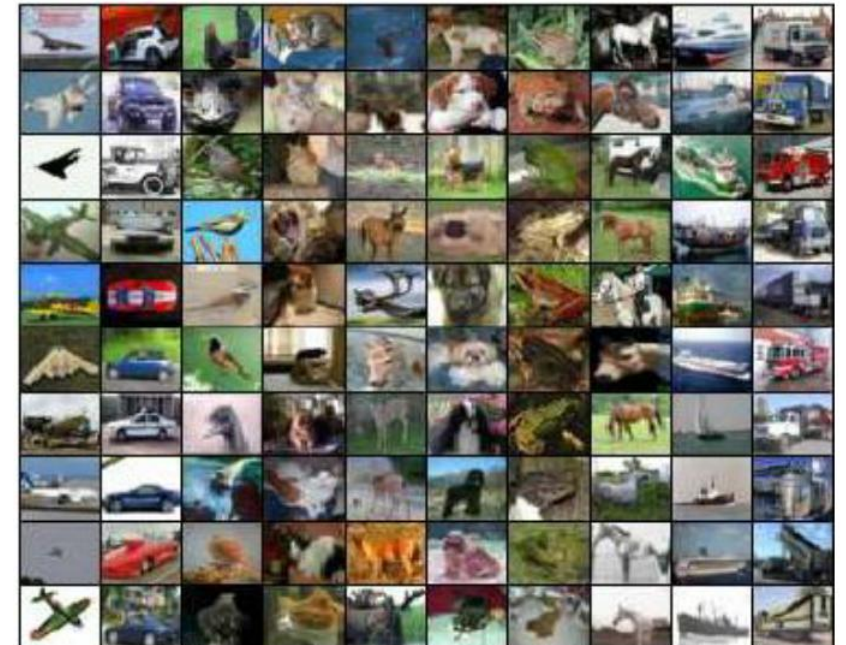
MNIST

- Handwritten digits
- 28x28 grayscale images
- 60,000 train images
- 10,000 test images



CIFAR10

- 10 categories (airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck)
- 32x32 color images
- 50,000 train images
- 10,000 test images



Datasets

MNIST

CIFAR10

IMAGENET

- 1,000 classes
- 256x256 color images
- ~1.2M images
- 2010-2017: ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

people



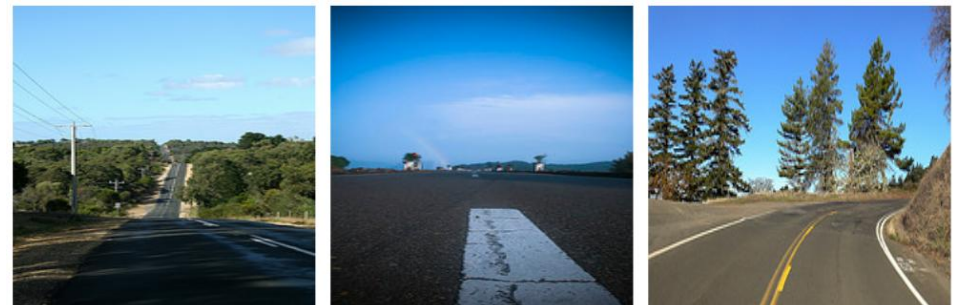
child



house



road



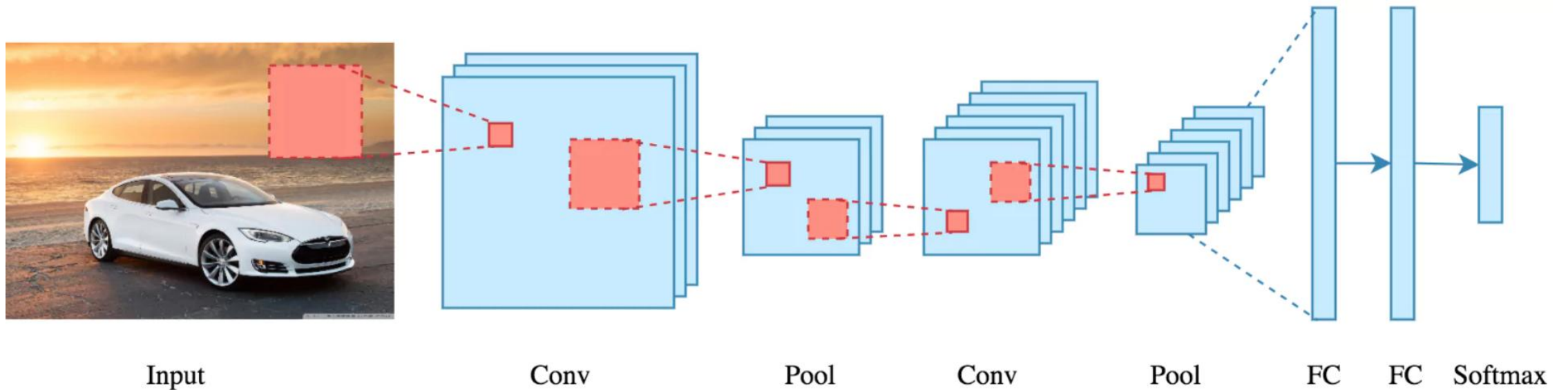
Data Augmentation

- Technique to artificially enlarge the training set.
- Examples: flips, rotations, crops, color jitter, noise.
- Apply simple transformations while keeping the label the same.
- Helps reduce overfitting and improve generalization.



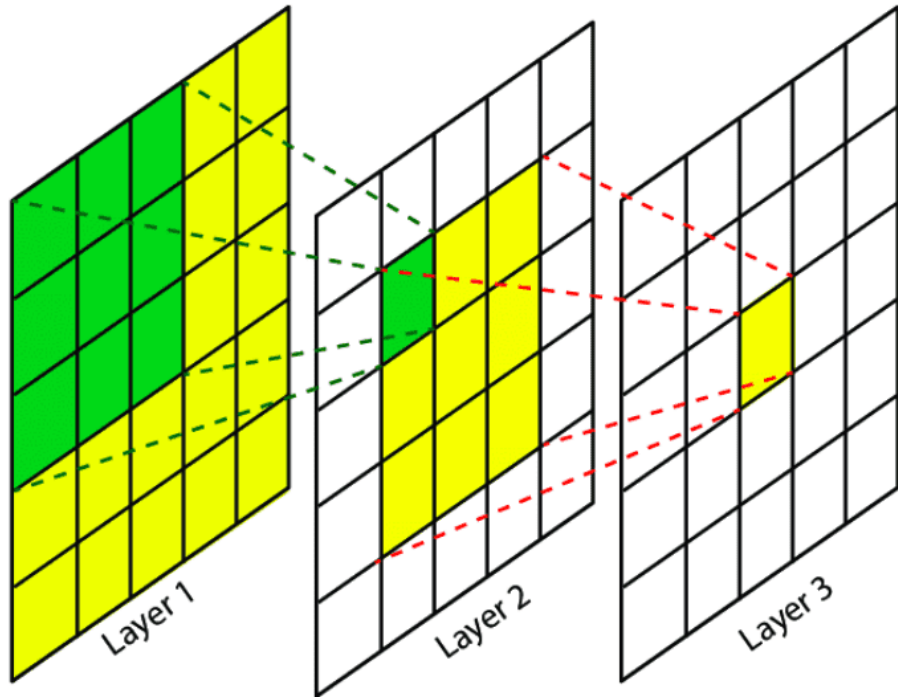
Convolutional neural network (CNN)

General term for neural networks that apply convolutions. Typically, they use convolutional layers in the early stages and linear (fully connected) layers in the final stages.



Receptive Field

The region of the input image that influences a neuron's activation.

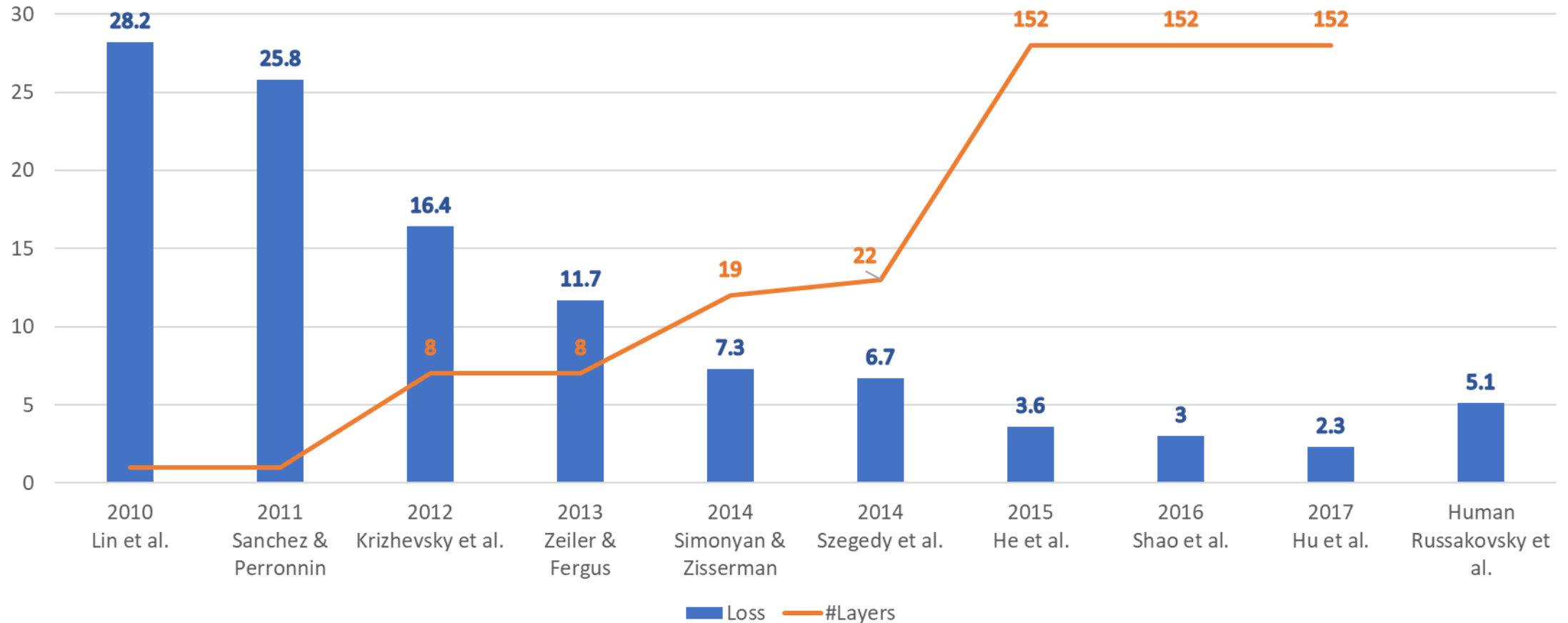


For example:

Here it's 5X5 because

$$1 \times 1 \leftarrow 3 \times 3 \leftarrow 5 \times 5$$

ImageNet Challenge



AlexNet (Krizhevsky et al. 2012)

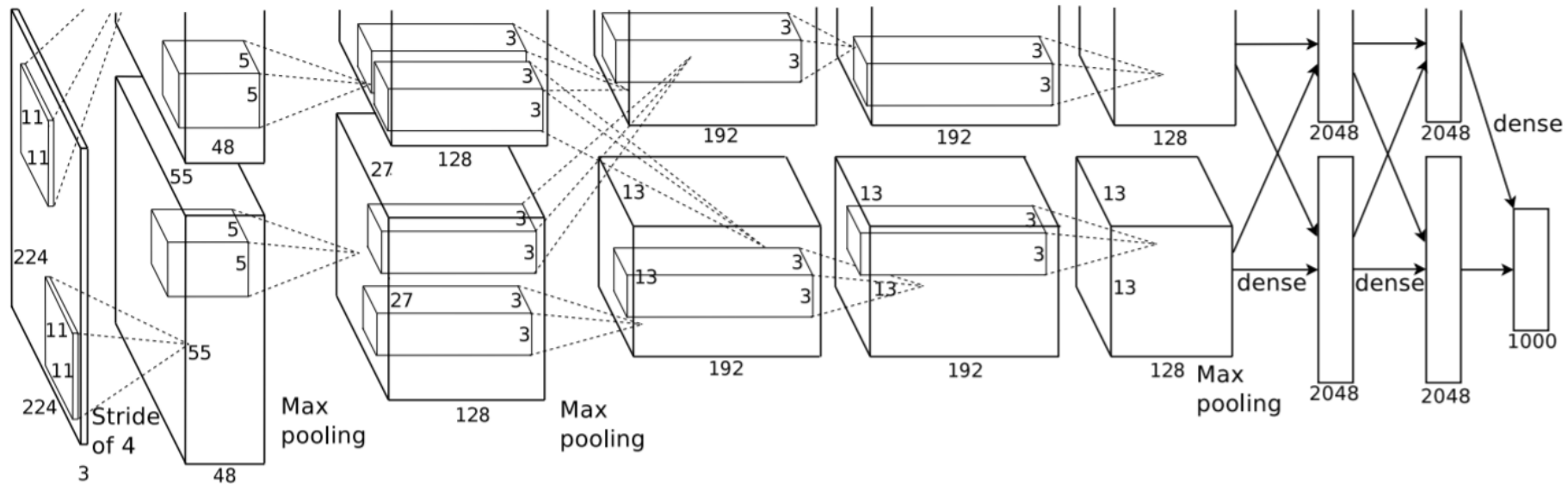
ImageNet Classification with Deep Convolutional Neural Networks

Alex Krizhevsky
University of Toronto
kriz@cs.utoronto.ca

Ilya Sutskever
University of Toronto
ilya@cs.utoronto.ca

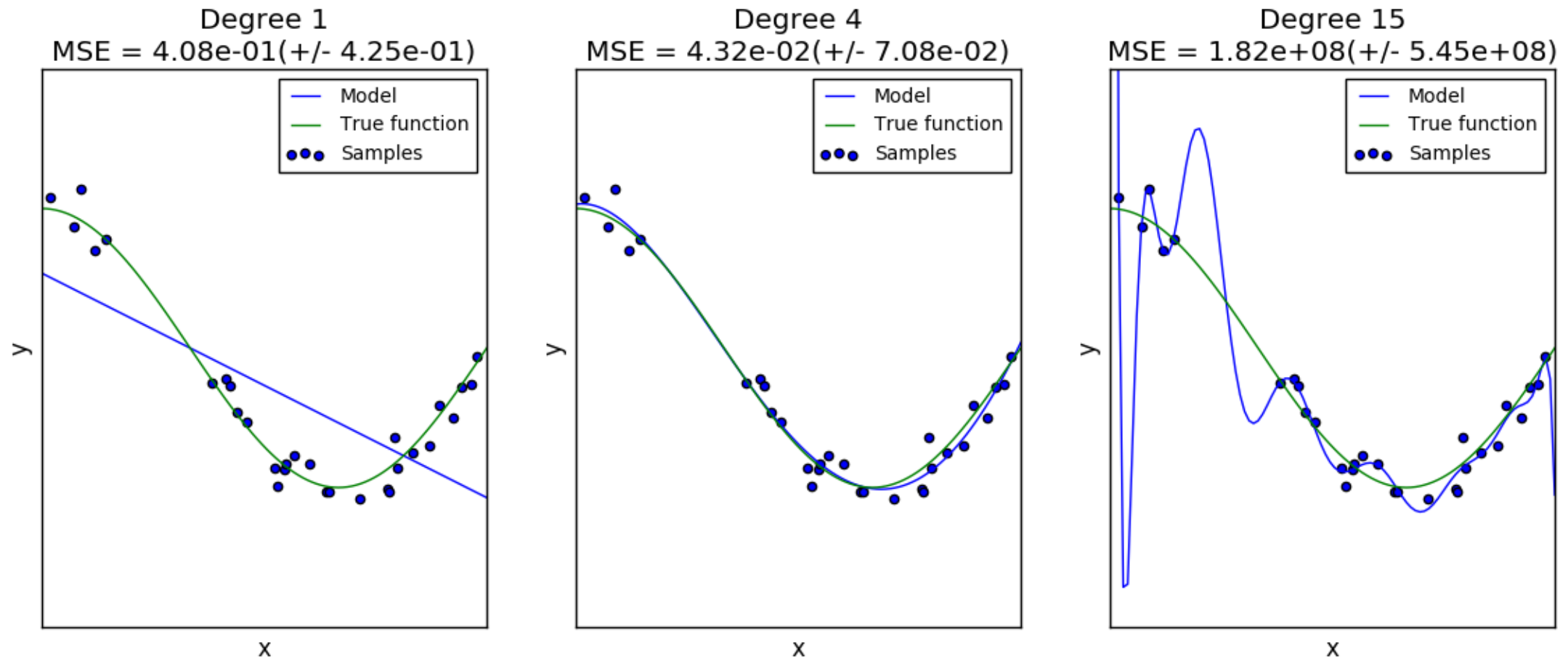
Geoffrey E. Hinton
University of Toronto
hinton@cs.utoronto.ca

91,548 citations



Dropout

Regularization technique to reduce overfitting



Dropout

Regularization technique to reduce overfitting

Randomly drop units during training

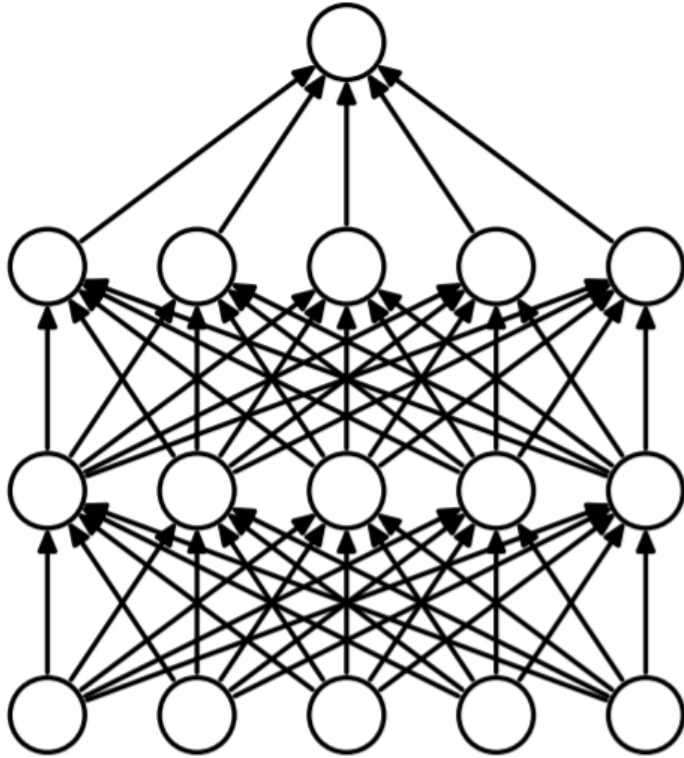
Prevents co-adaptions

- Units are trained according to a loss that is dependent on all units (what all units are doing)
- Units may change to “fix” mistakes of other units
- Builds complex co-adaptions that do not generalize to unseen data

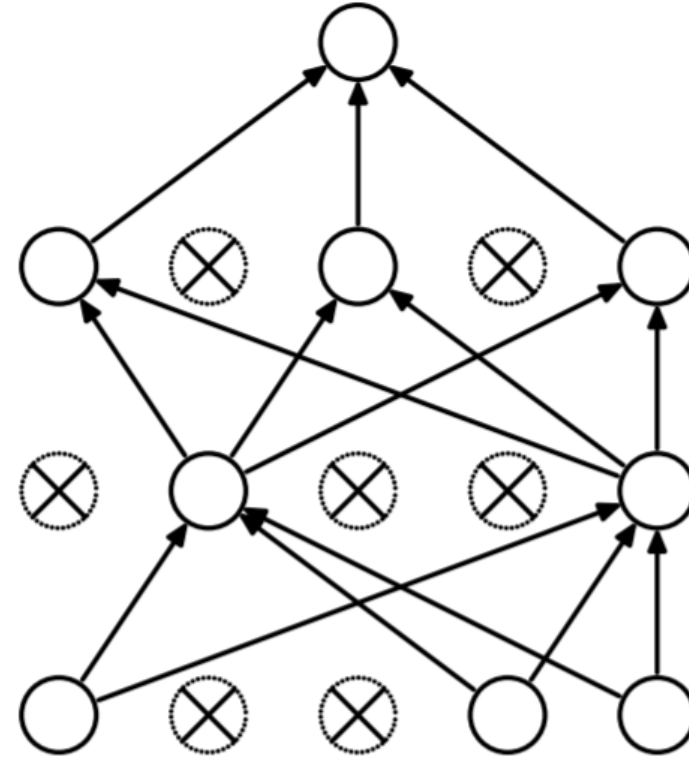
Model combination improves performance

- Dropout approximates combining many different models

Dropout

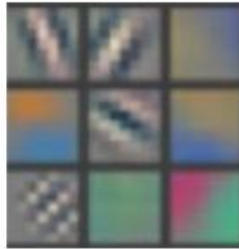


(a) Standard Neural Net

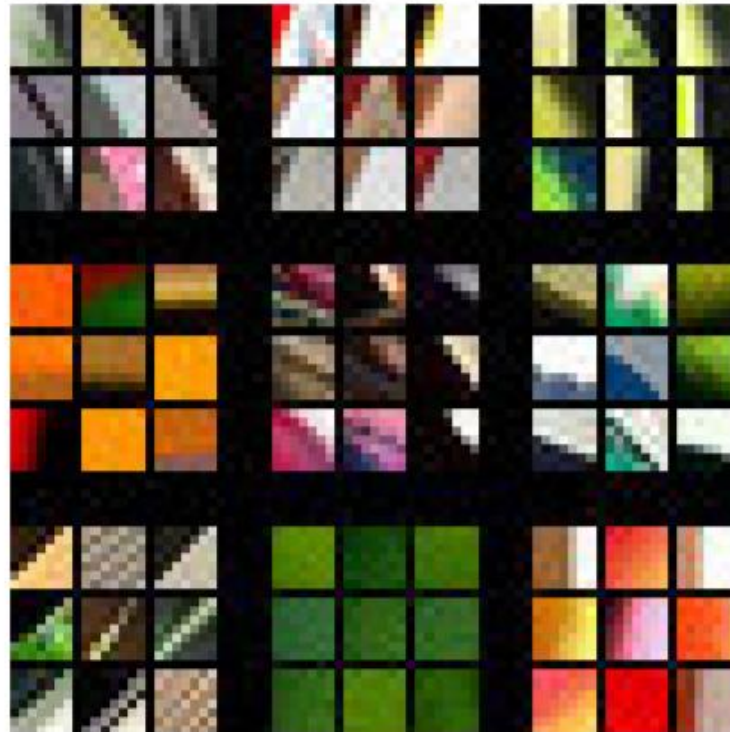


(b) After applying dropout.

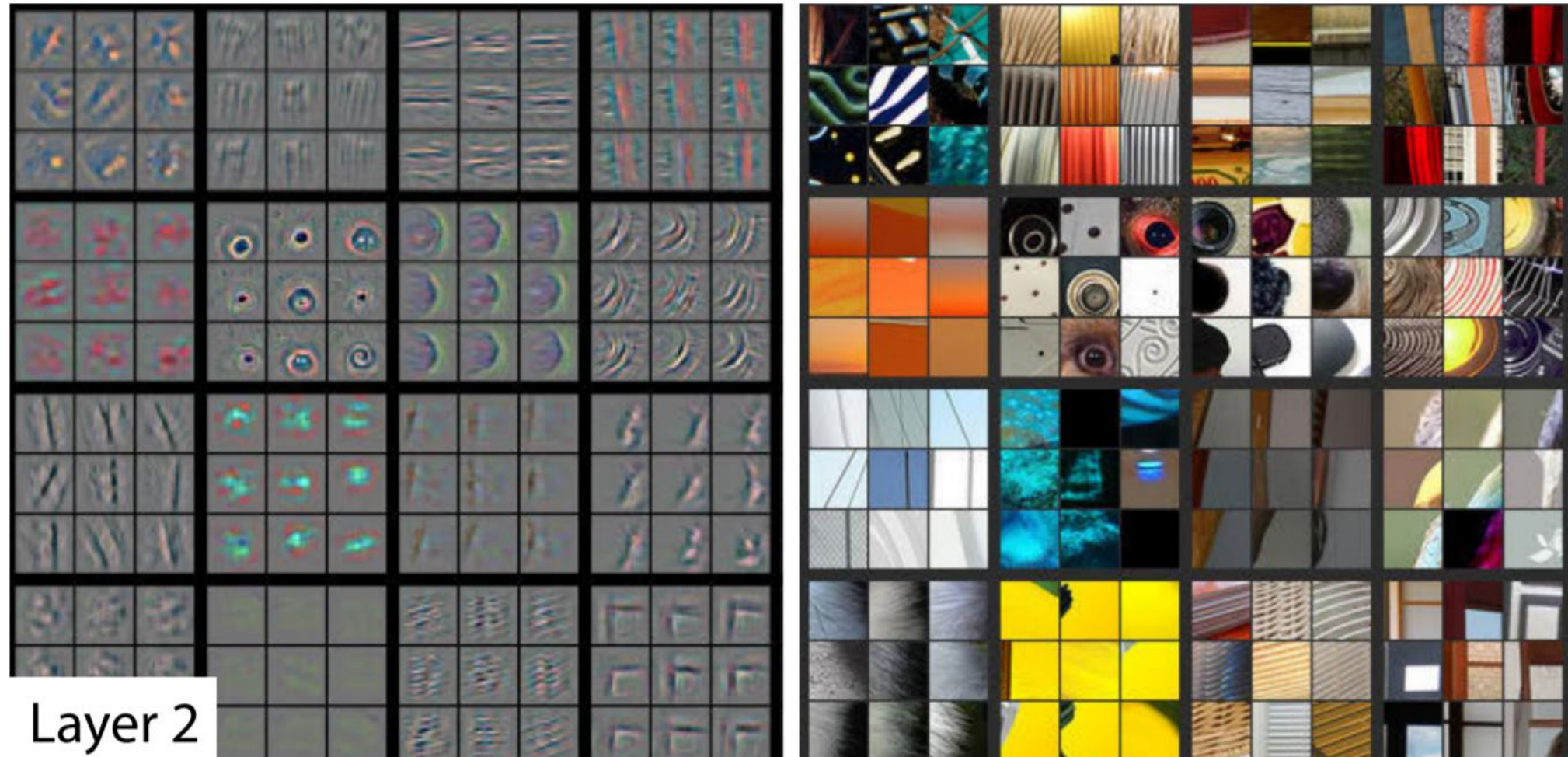
ZFNet (Zeiler and Fergus, 2013)



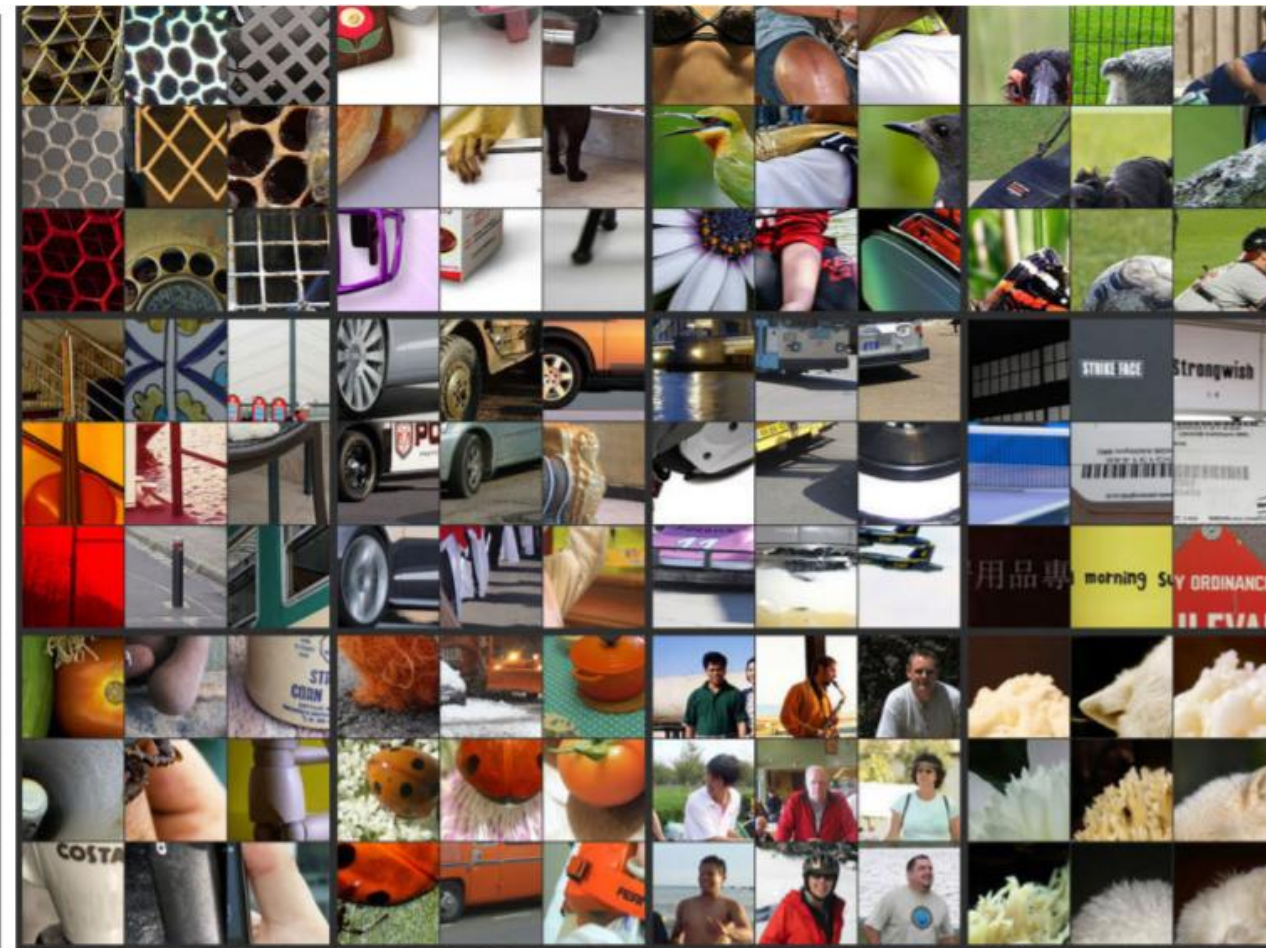
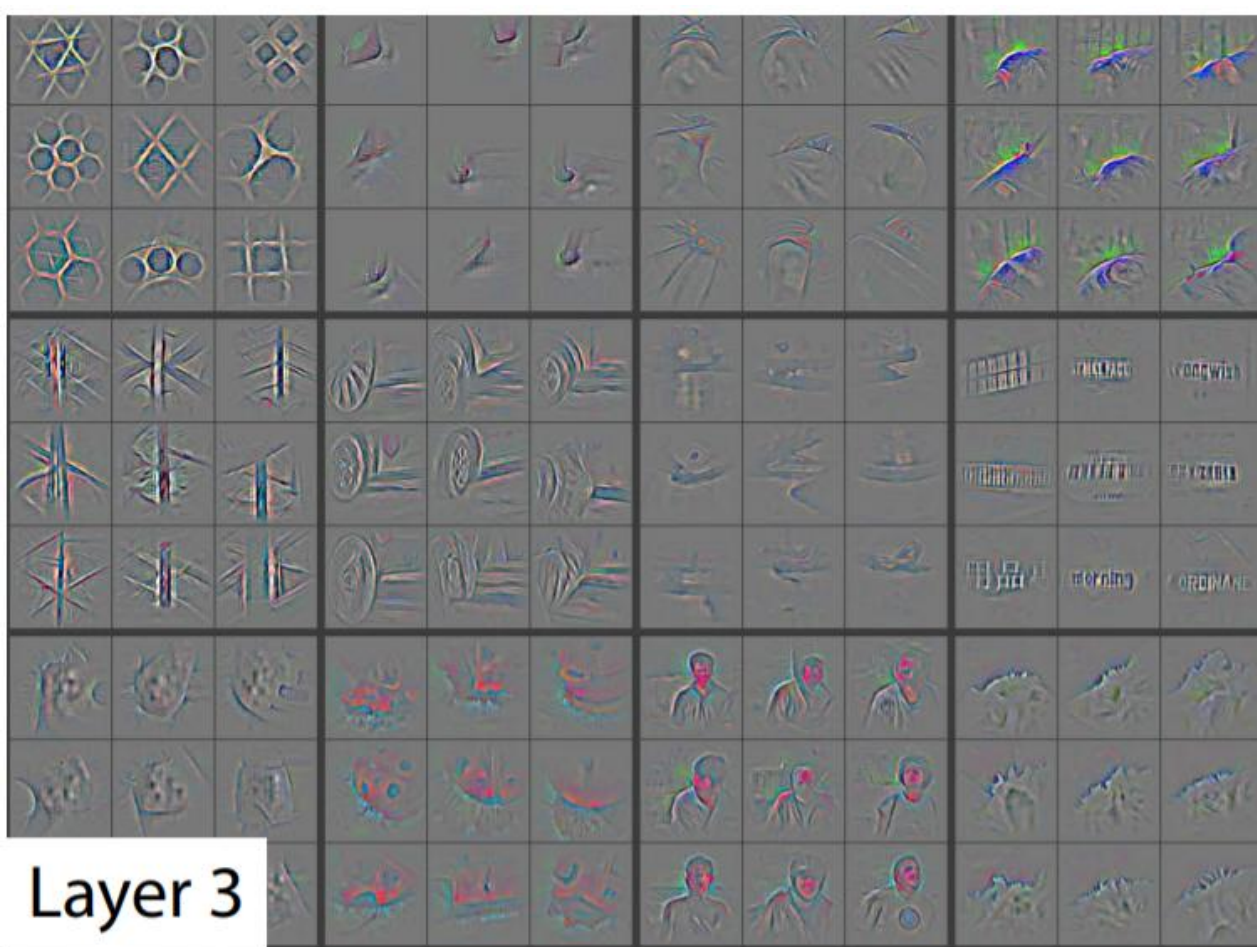
Layer 1



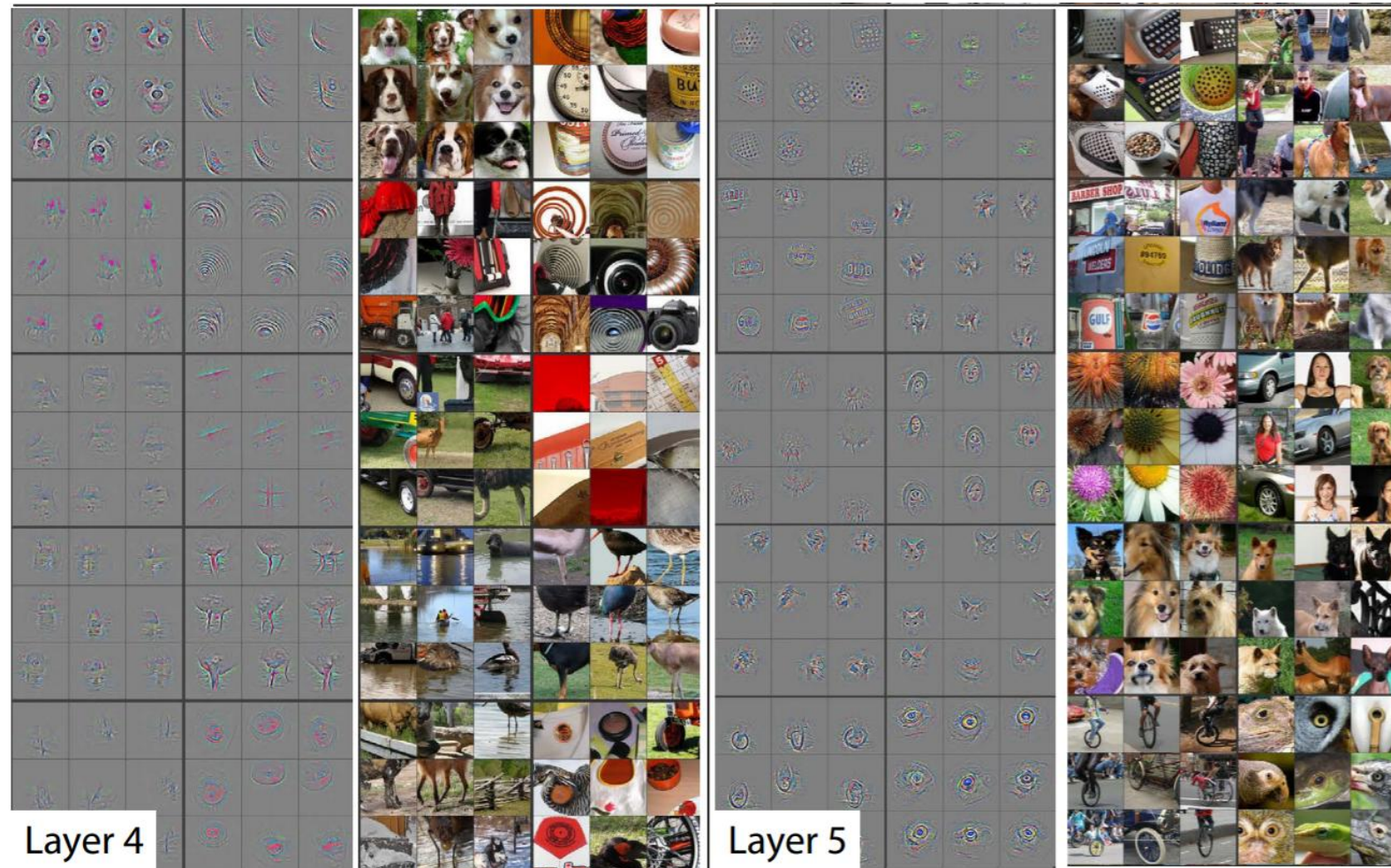
ZFNet (Zeiler and Fergus, 2013)



ZFNet (Zeiler and Fergus, 2013)



ZFNet (Zeiler and Fergus, 2013)



VGG (Simonyan and Zisserman, 2014)

VERY DEEP CONVOLUTIONAL NETWORKS FOR LARGE-SCALE IMAGE RECOGNITION

Karen Simonyan* & Andrew Zisserman⁺

Visual Geometry Group, Department of Engineering Science, University of Oxford
`{karen, az}@robots.ox.ac.uk`

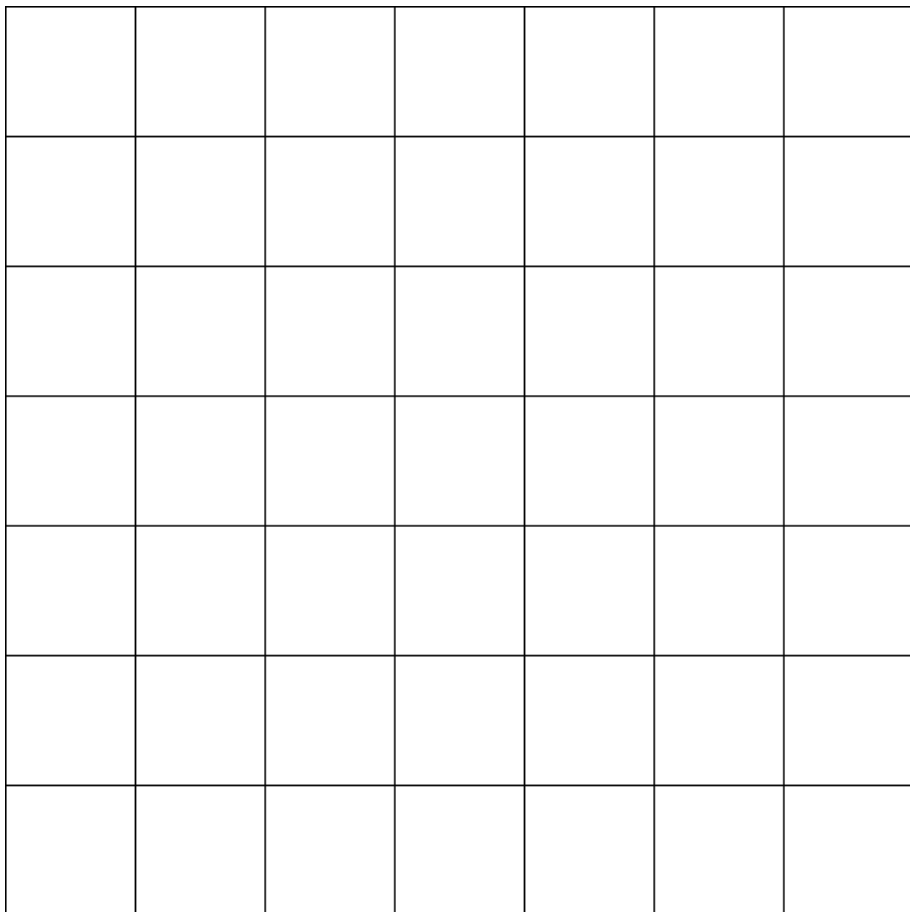
VGG (Simonyan and Zisserman, 2014)

Deeper networks

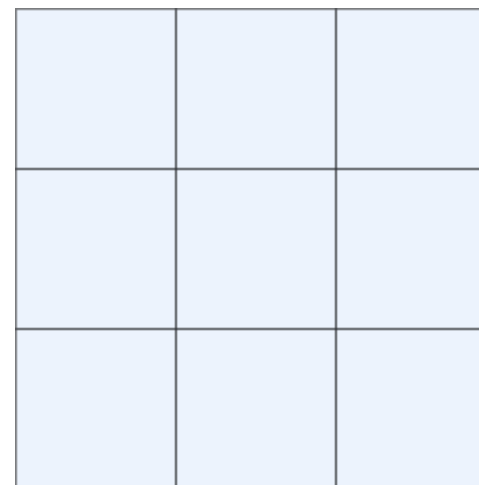
- 8 layers (AlexNet) → 16-19 layers (VGG16\VGG19)

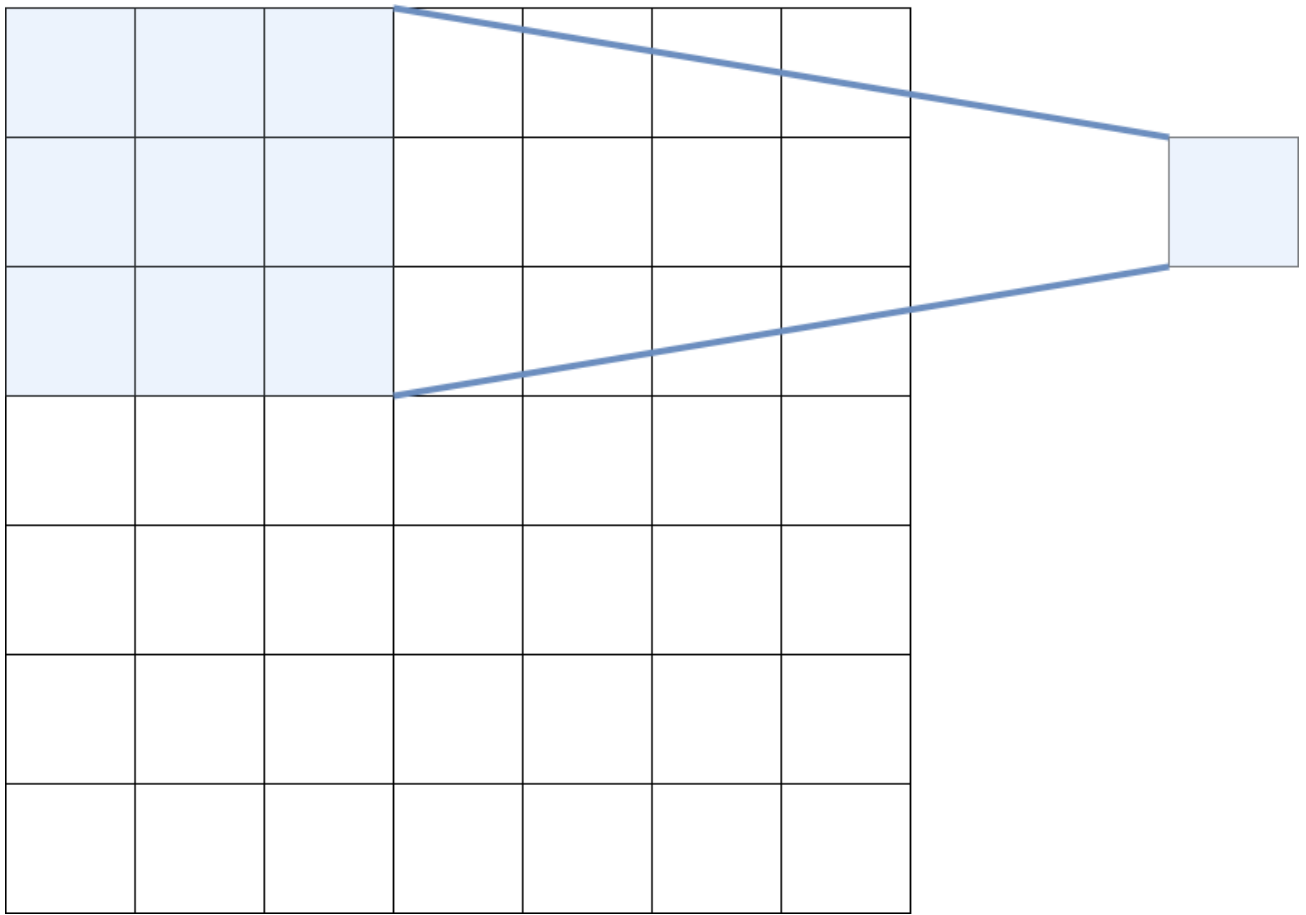
Smaller conv kernels: 3x3 instead of 7x7

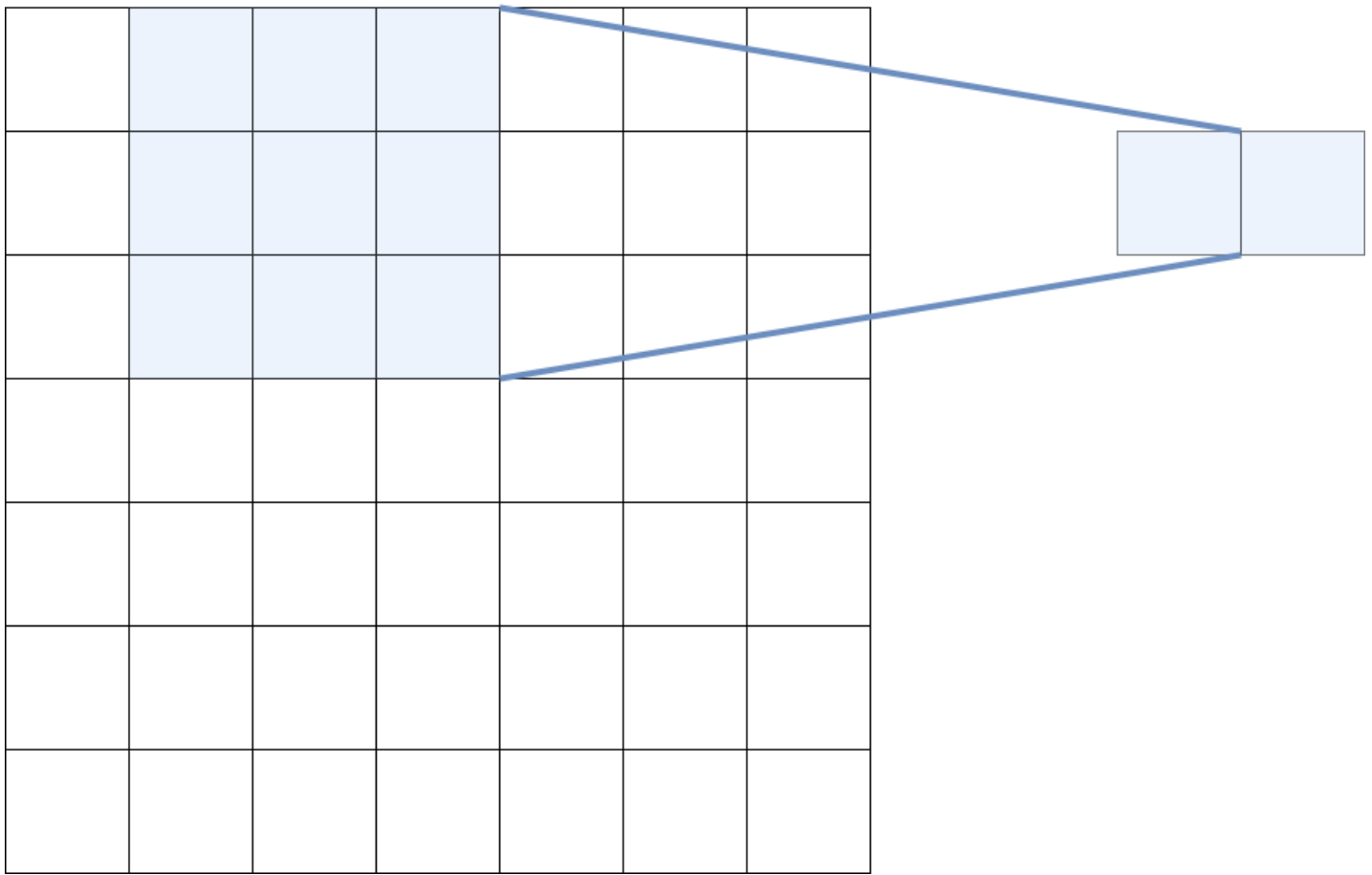
- Stacking three 3x3 conv has the same receptive field as one 7x7 conv

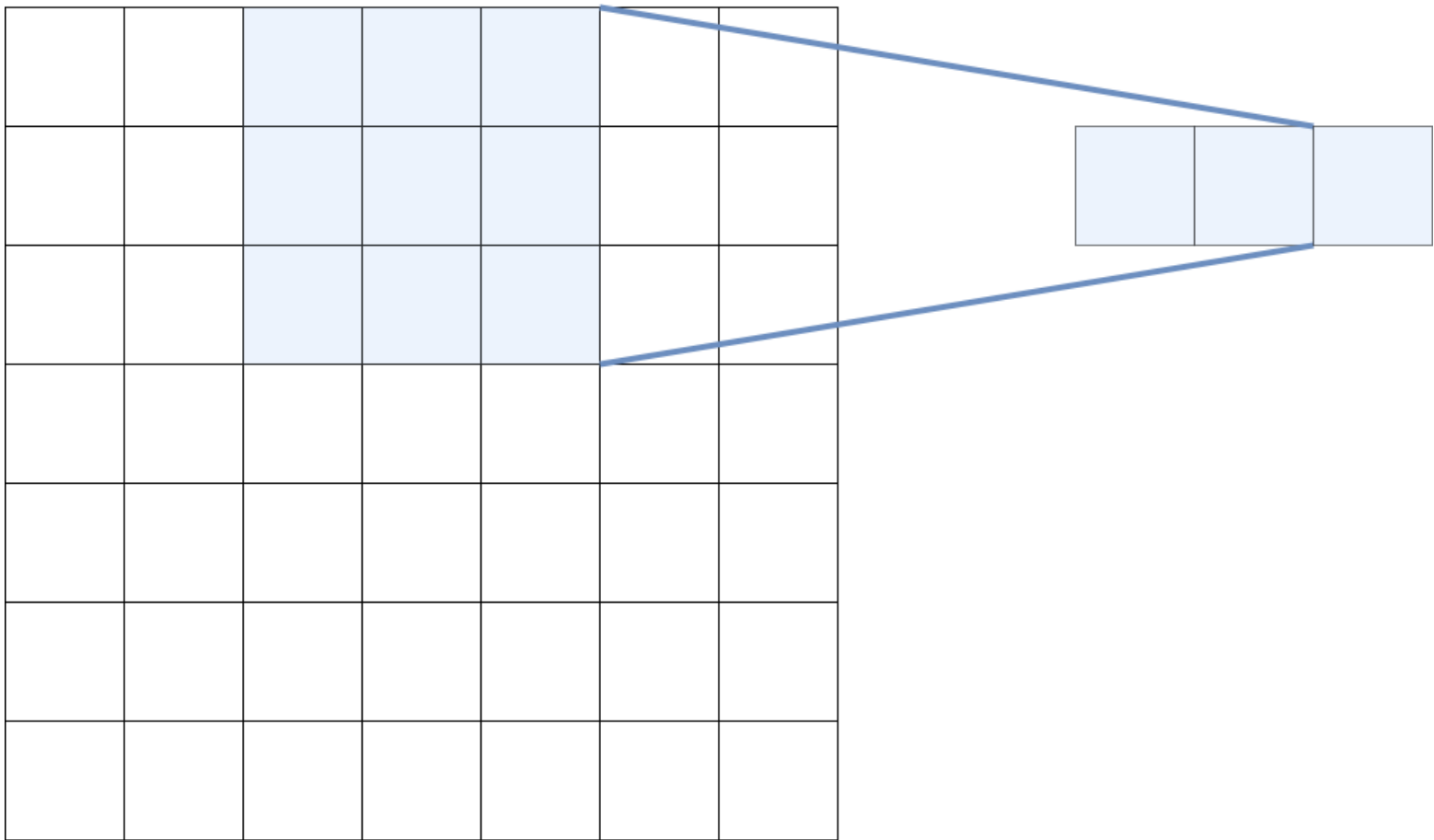


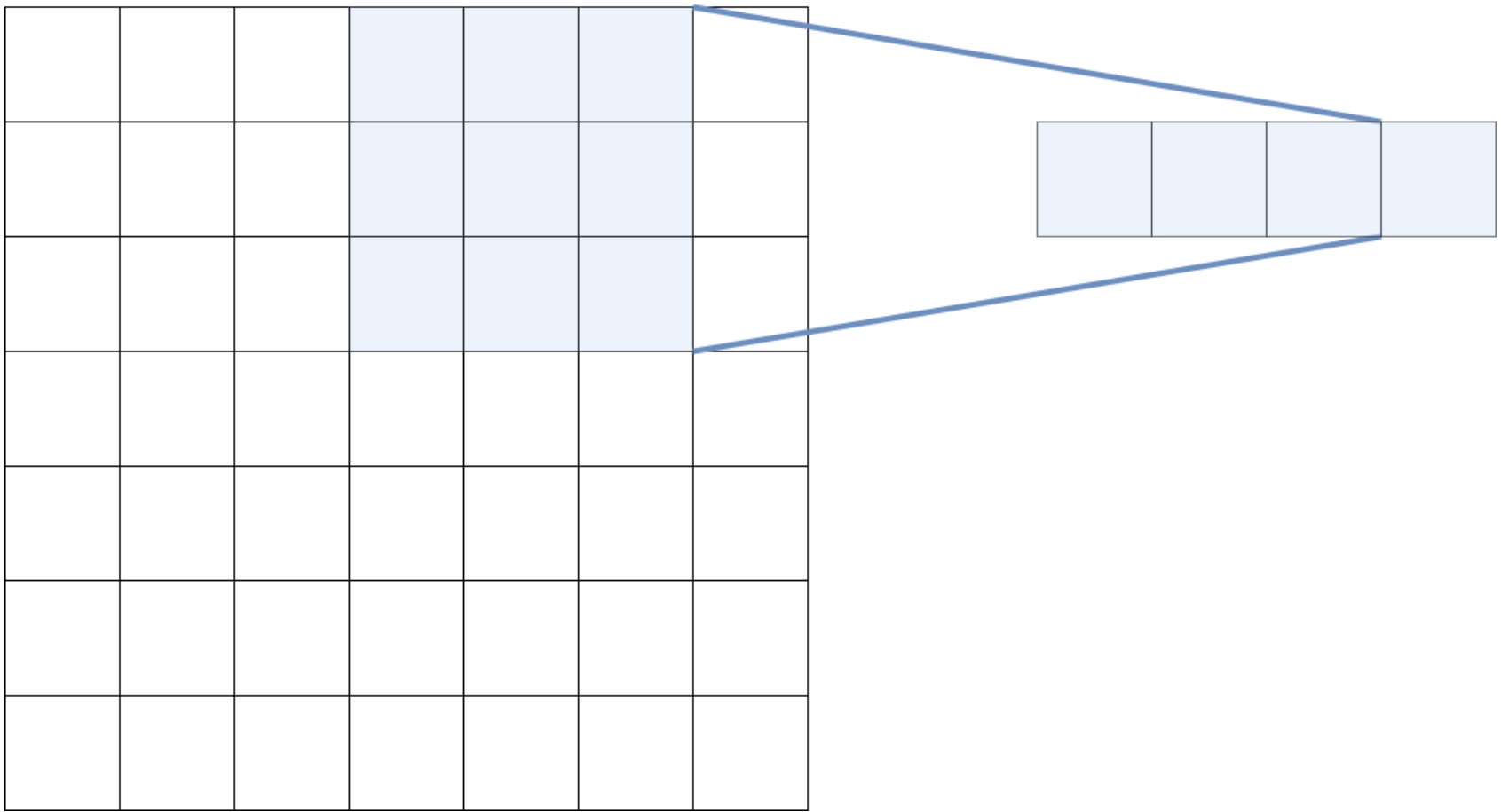
Kernel

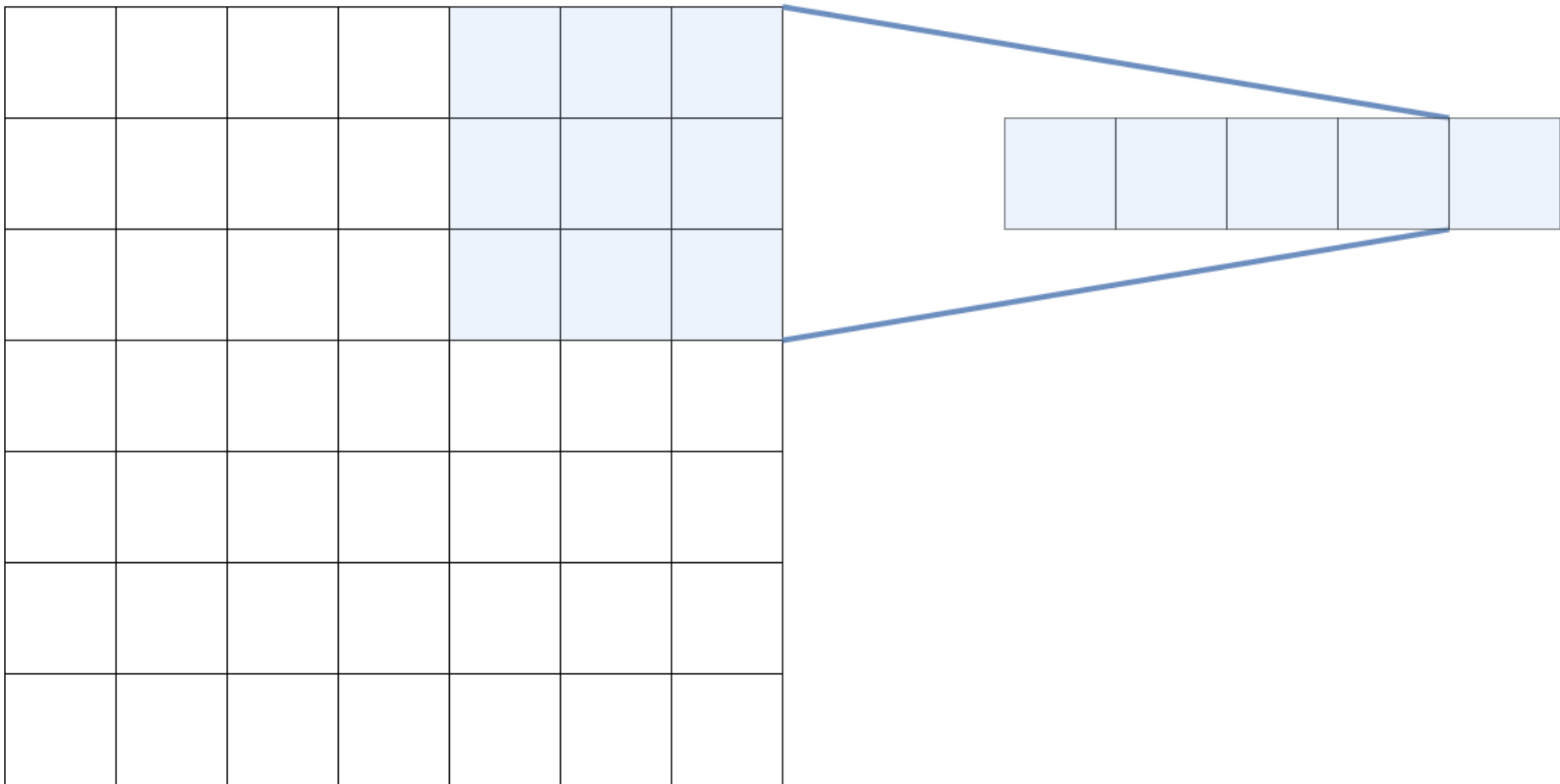


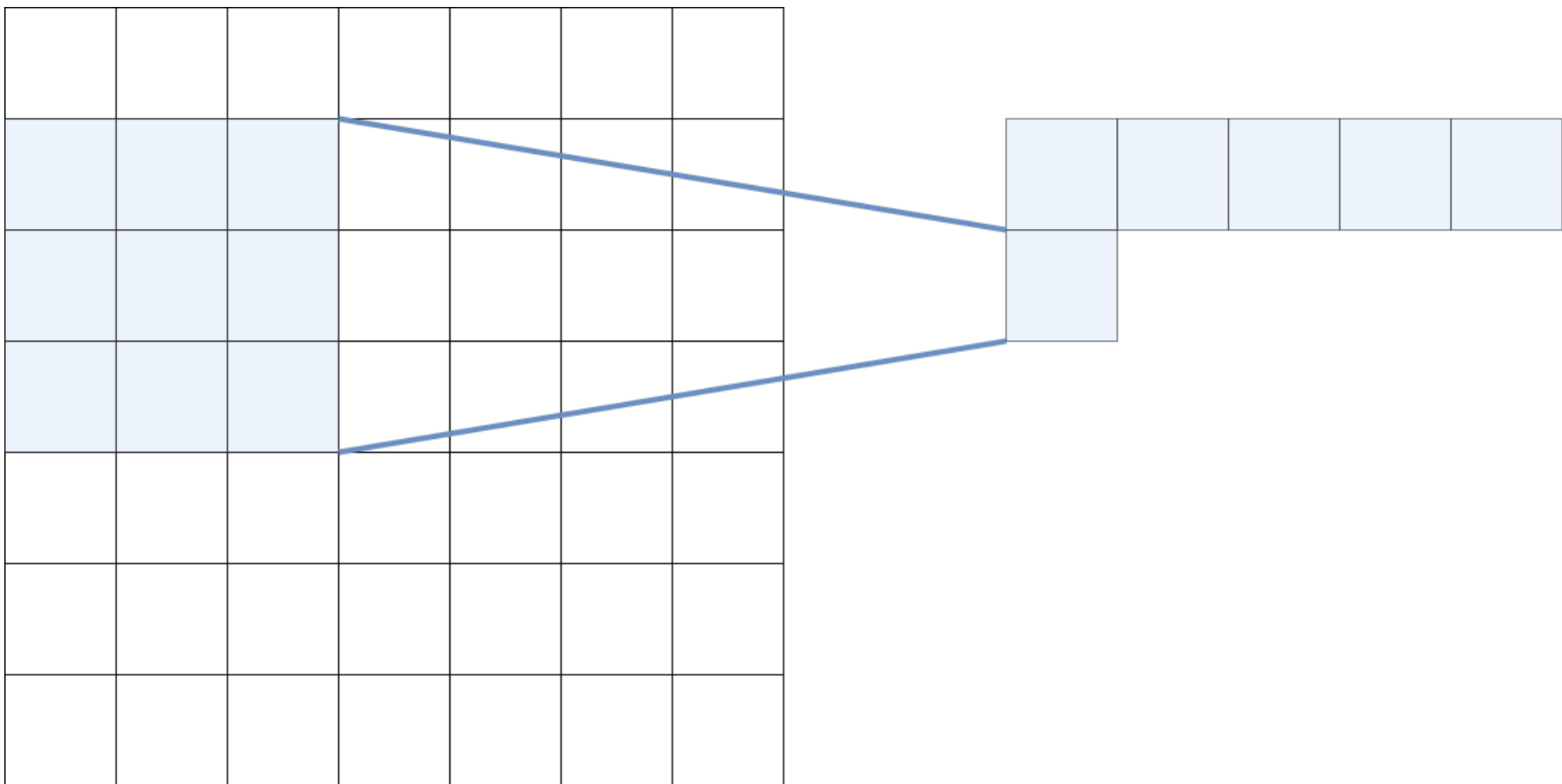


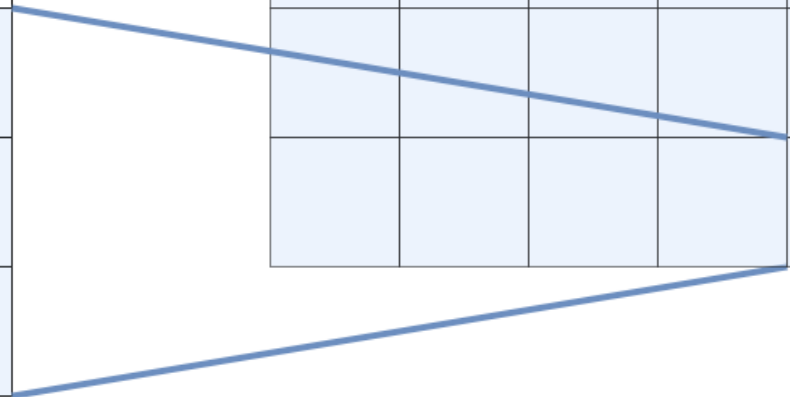
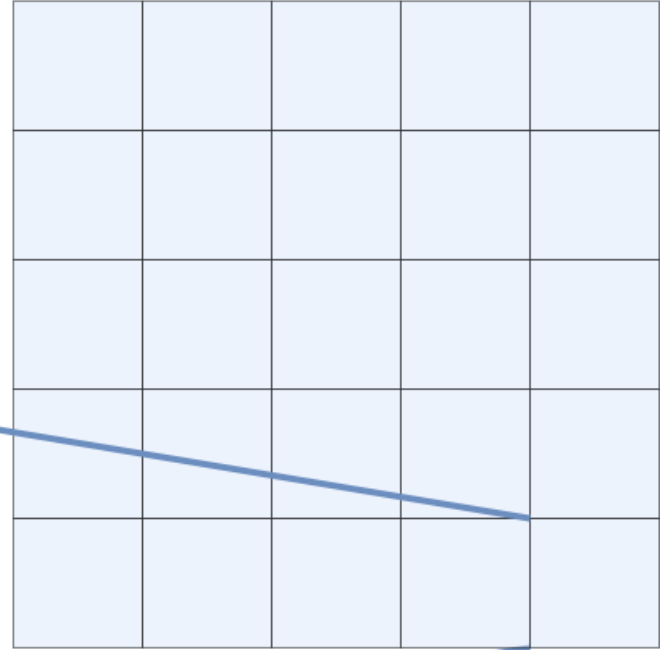
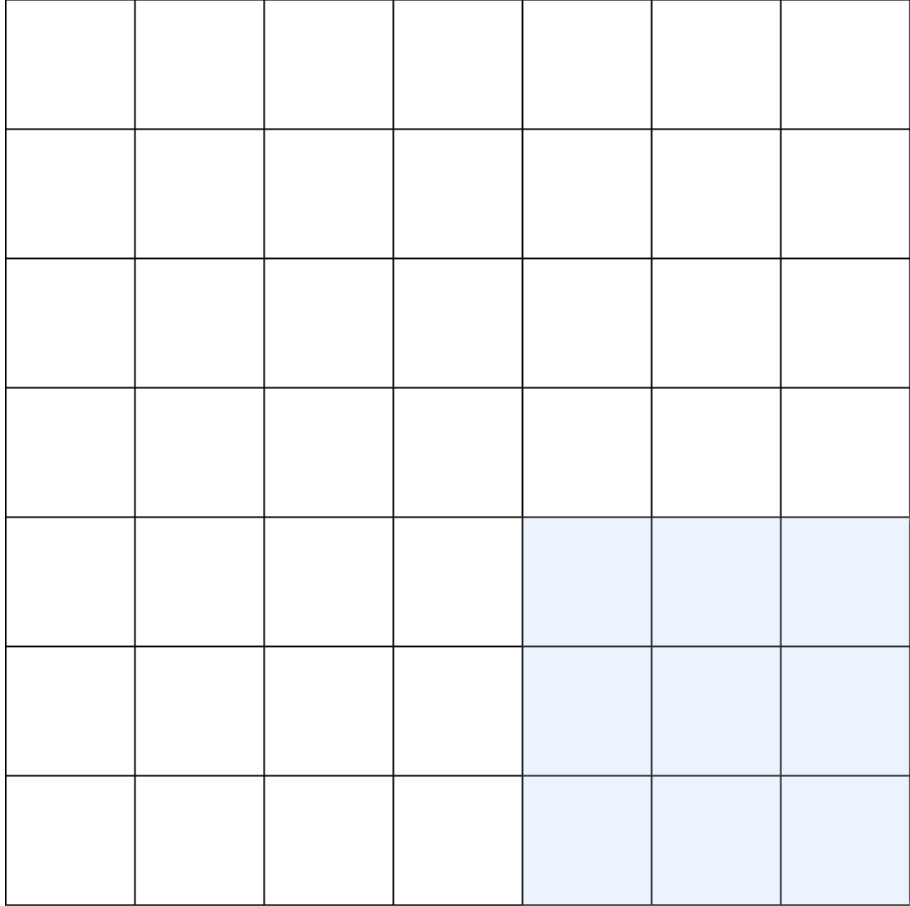


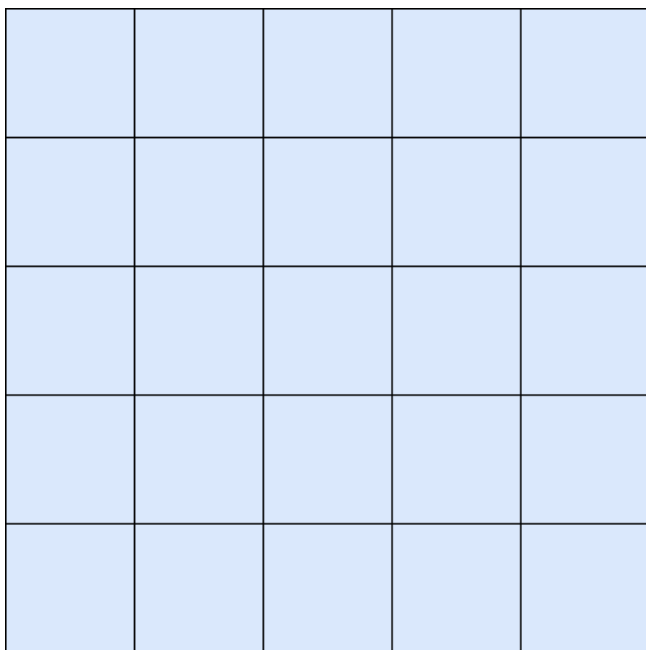




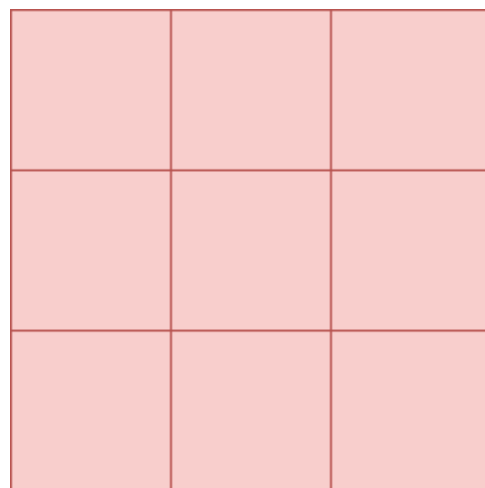


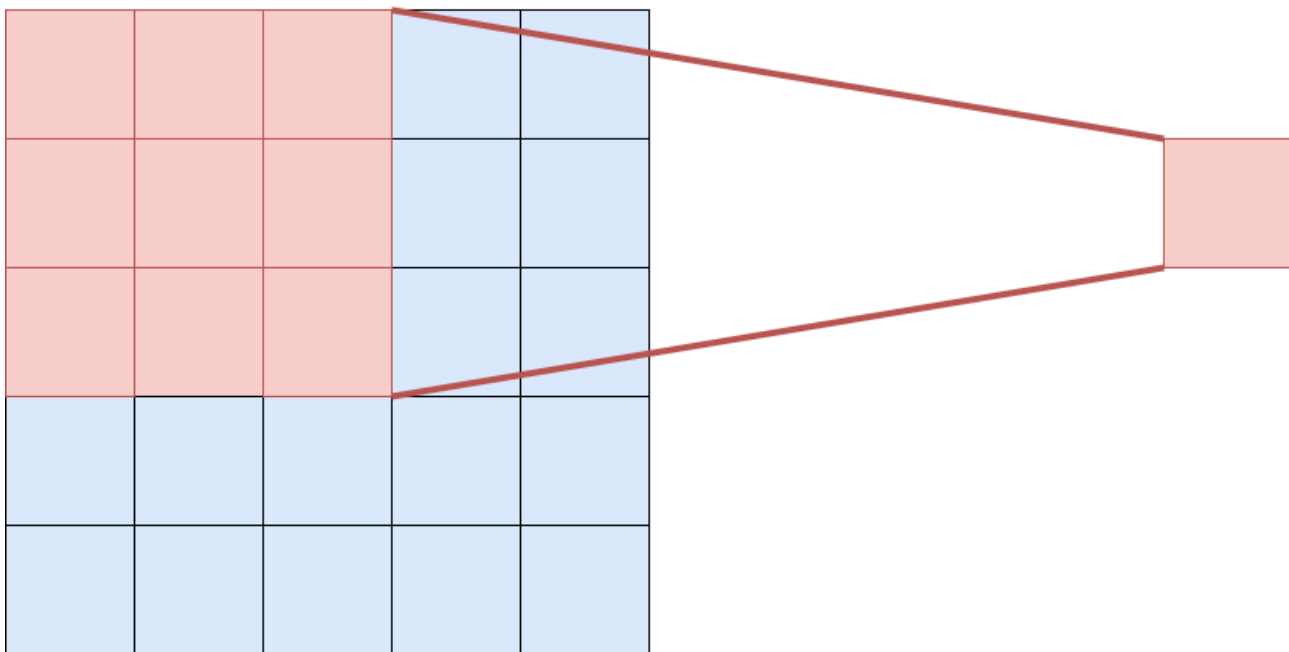


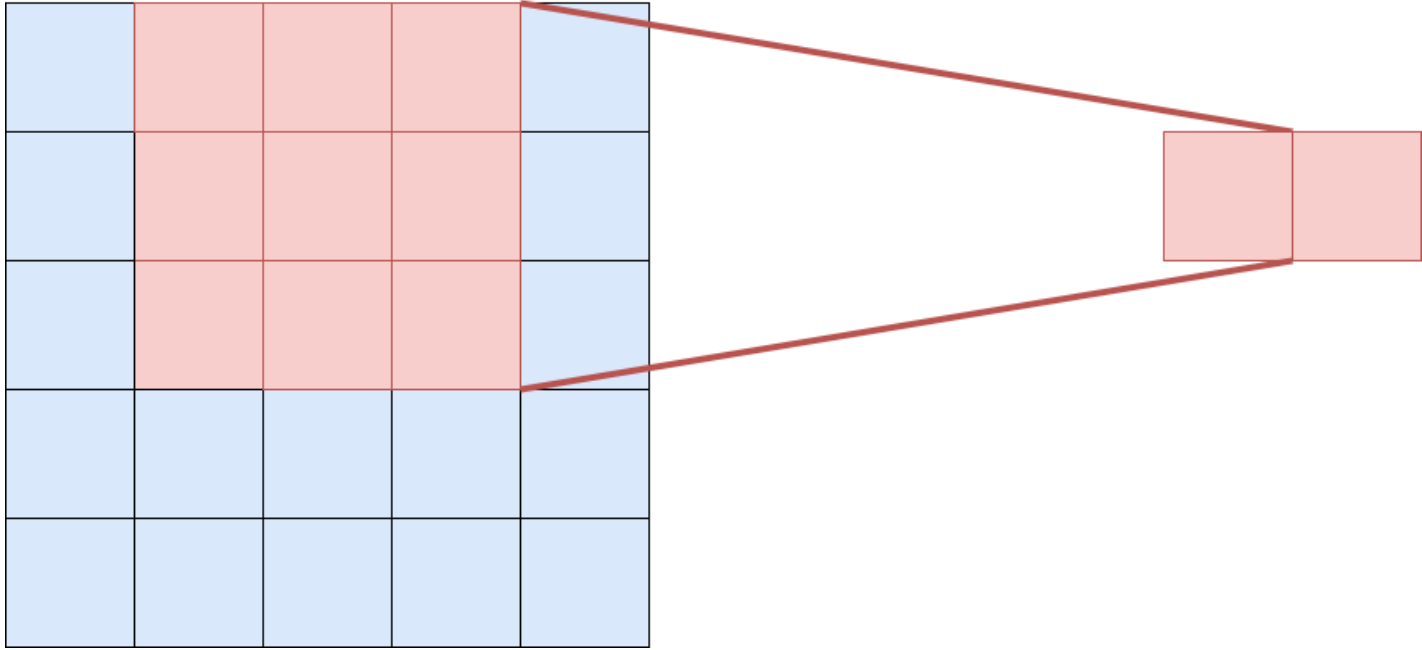


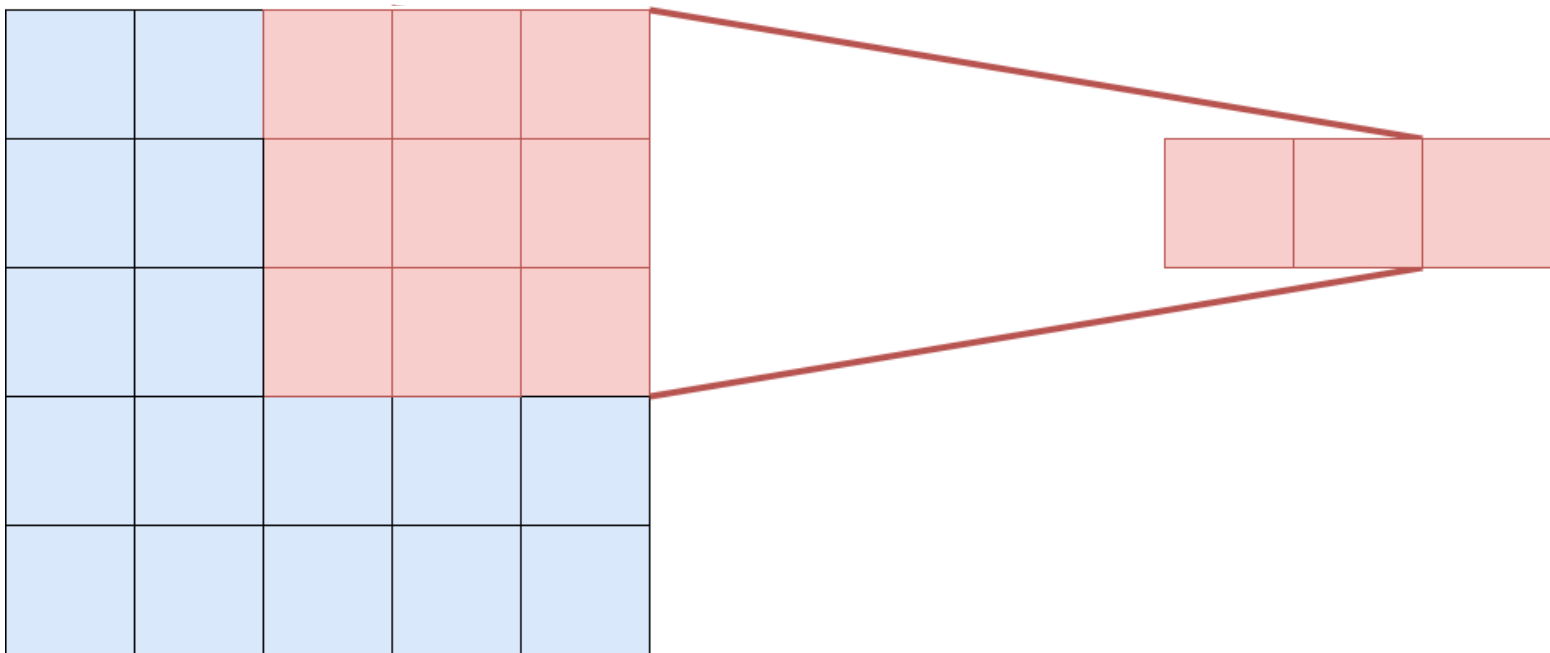


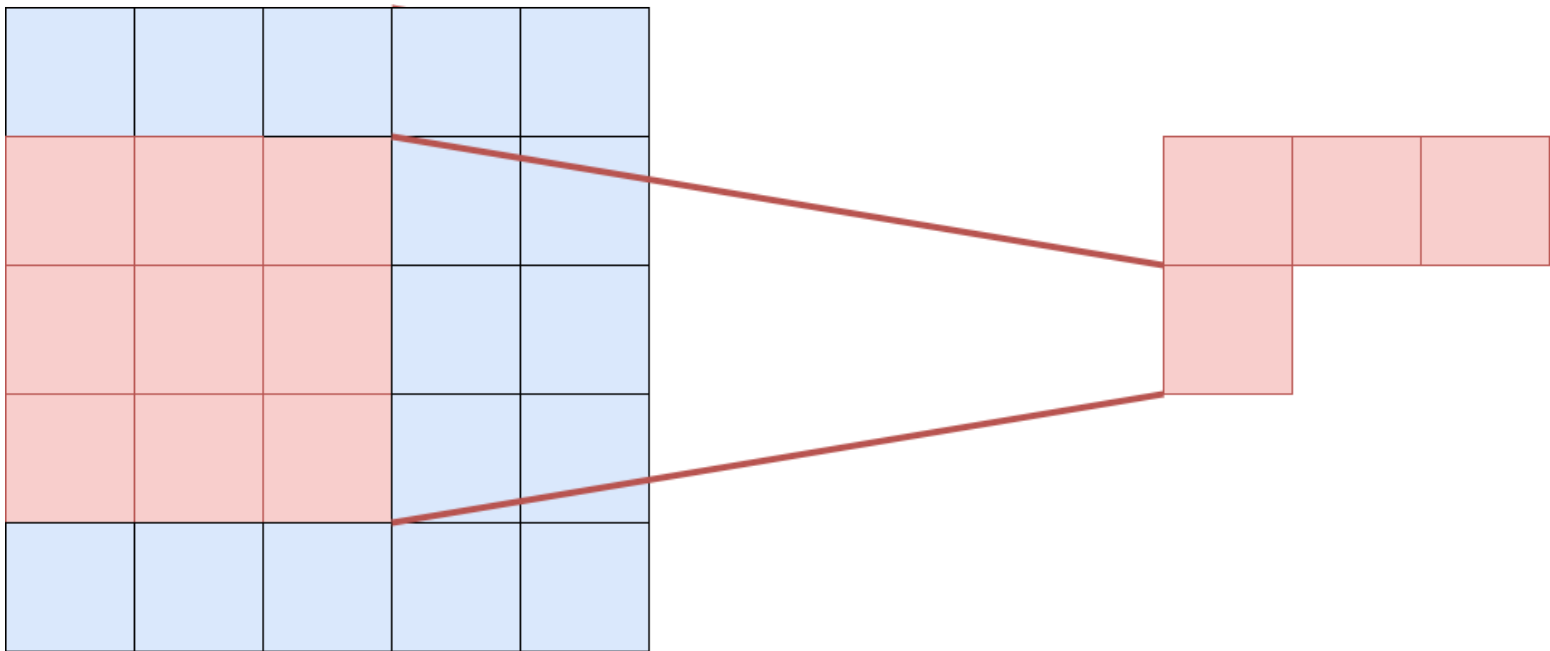
Kernel

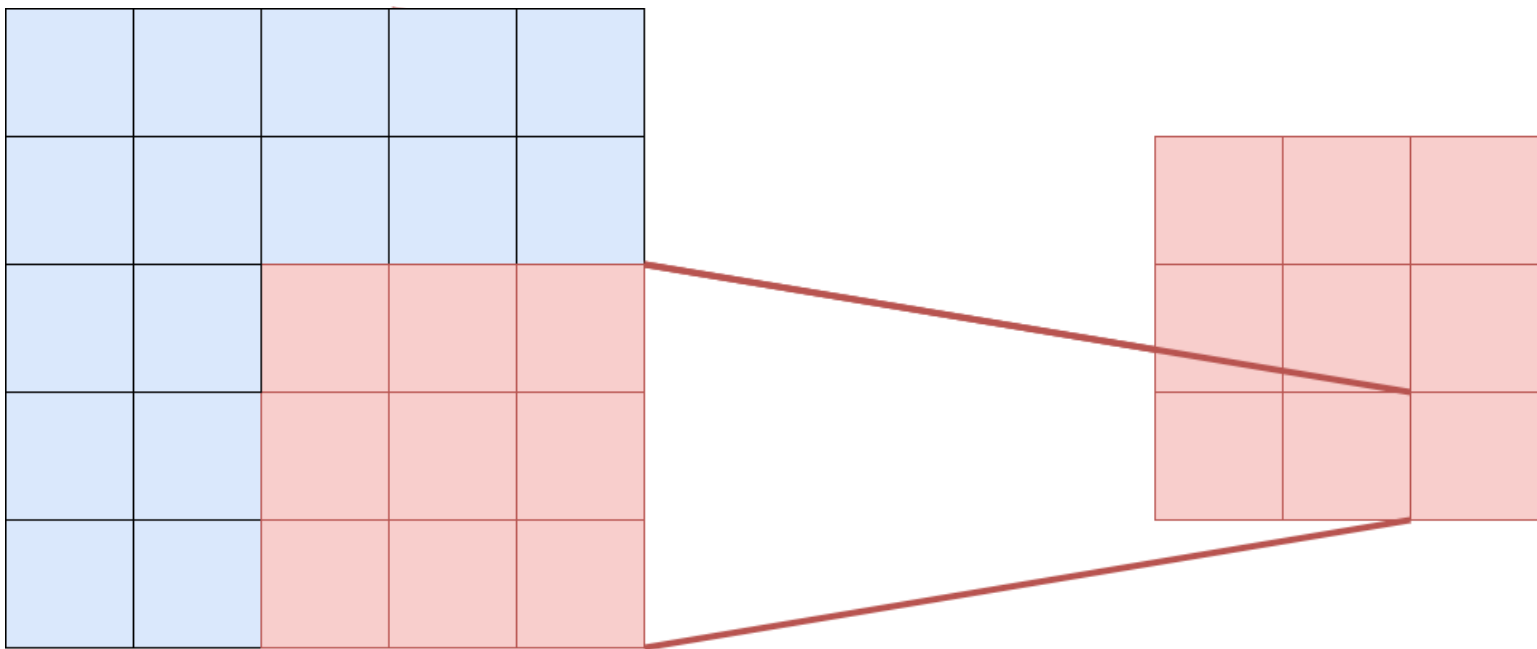


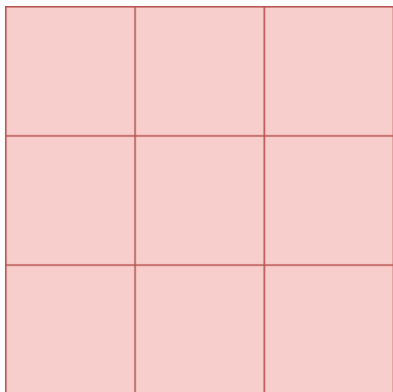




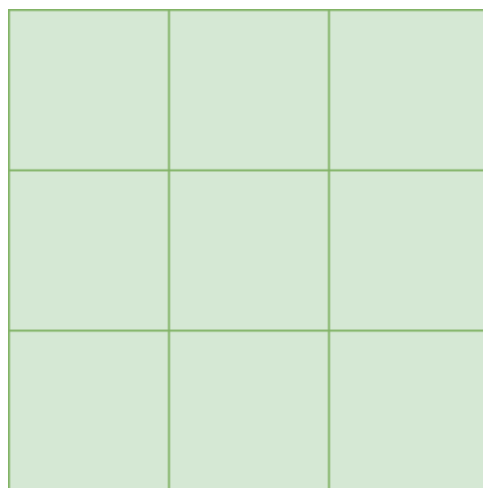


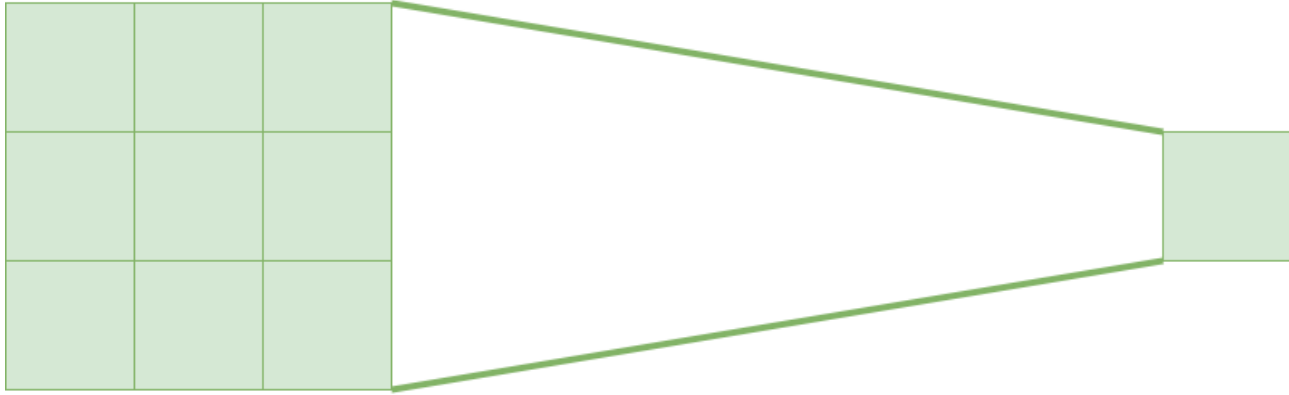


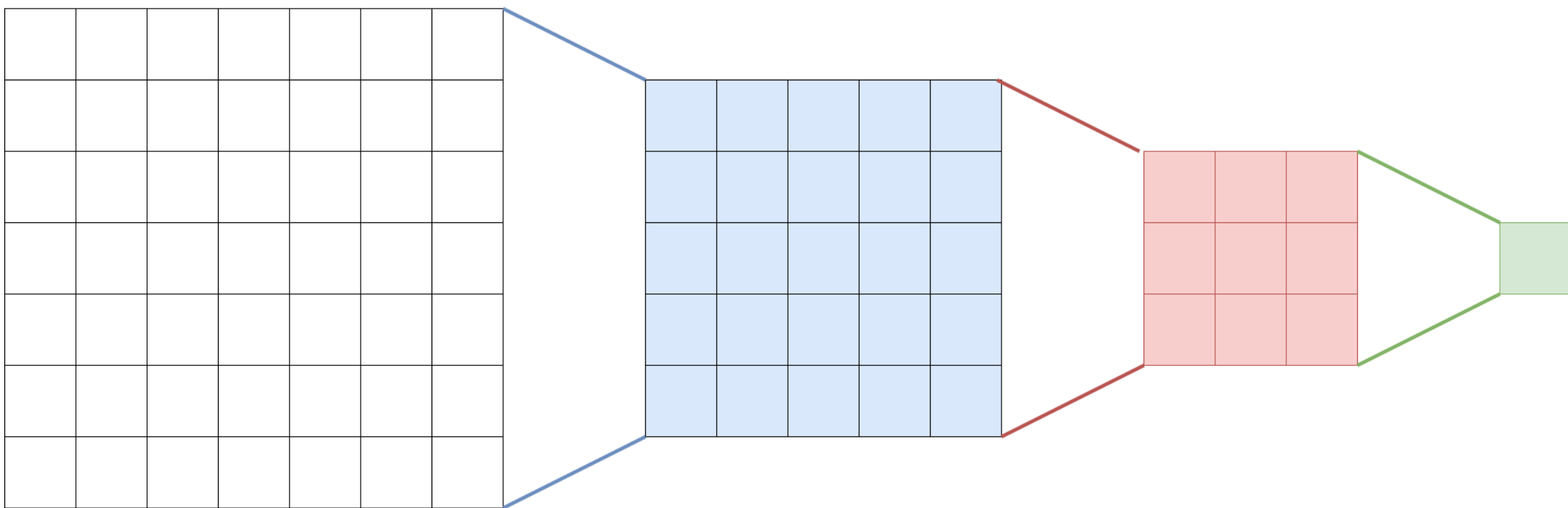




Kernel







VGG (Simonyan and Zisserman, 2014)

Deeper networks

- 8 layers (AlexNet) → 16-19 layers (VGG16\VGG19)

Smaller conv kernels: 3x3 instead of 7x7

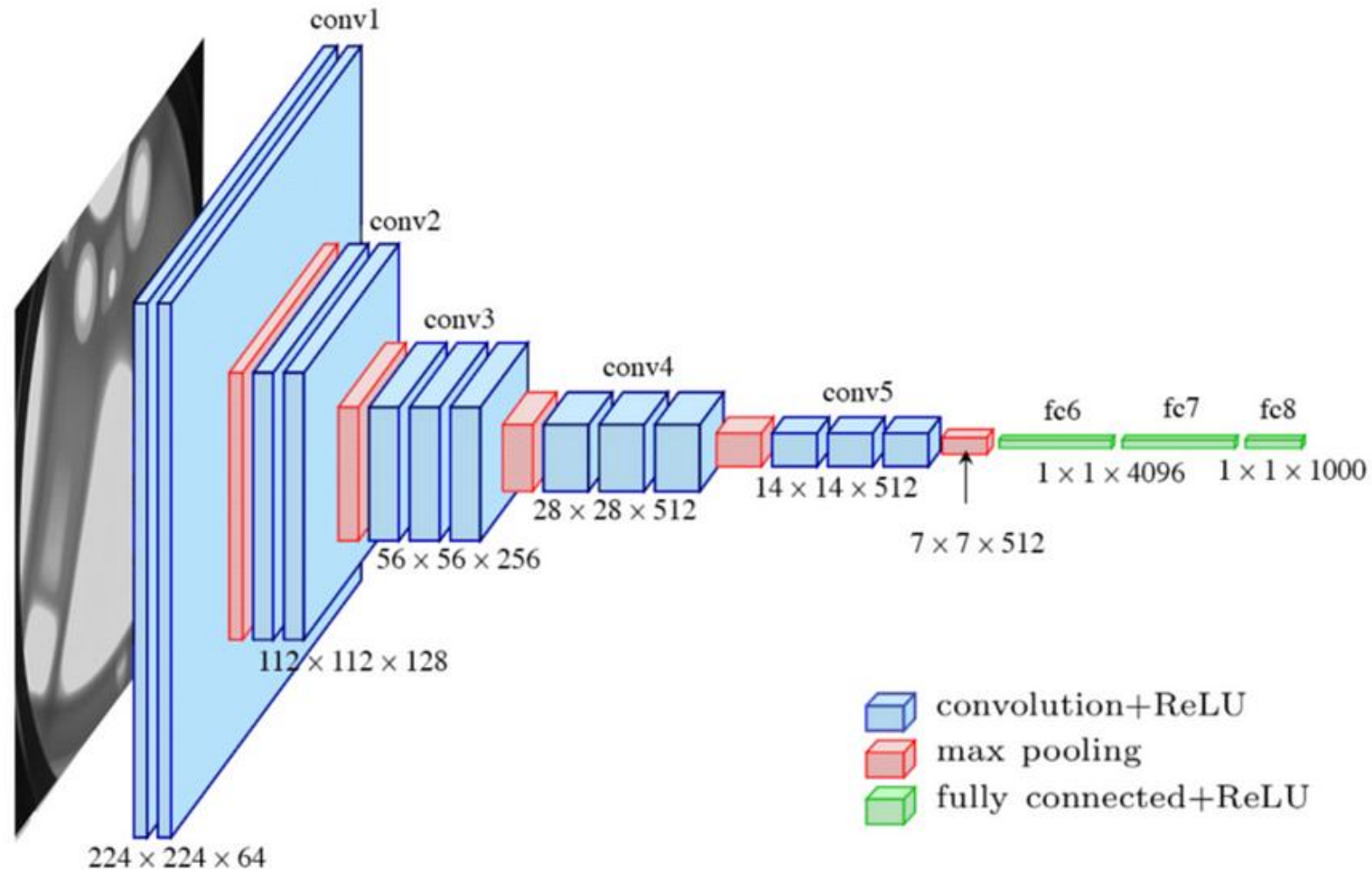
- Stacking three 3x3 conv has the same receptive field as one 7x7 conv
- More non linearities (deeper)
- Less parameters:

$3 \cdot (3^2 C^2) = 27C^2$ VS $7^2 C^2 = 49C^2$ (C is the number of input and output channels) → 81% more parameters!

No local response normalization

Good basis for transfer learning

VGG (Simonyan and Zisserman, 2014)



GoogLeNet (Szegedy et al., 2014)

Going deeper with convolutions

Christian Szegedy

Google Inc.

Wei Liu

University of North Carolina, Chapel Hill

Yangqing Jia

Google Inc.

Pierre Sermanet

Google Inc.

Scott Reed

University of Michigan

Dragomir Anguelov

Google Inc.

Dumitru Erhan

Google Inc.

Vincent Vanhoucke

Google Inc.

Andrew Rabinovich

Google Inc.

GoogLeNet (Szegedy et al., 2014)

Deeper networks: 22 layers

Computational efficiency

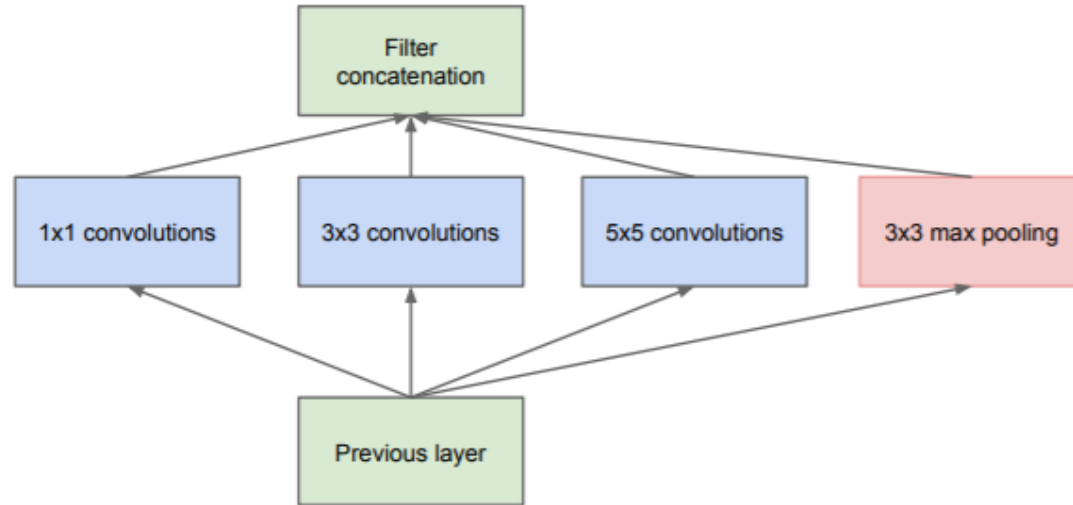
No fully connected

Less parameters: 5M parameters

- AlexNet: 60M
- VGG16: 138M

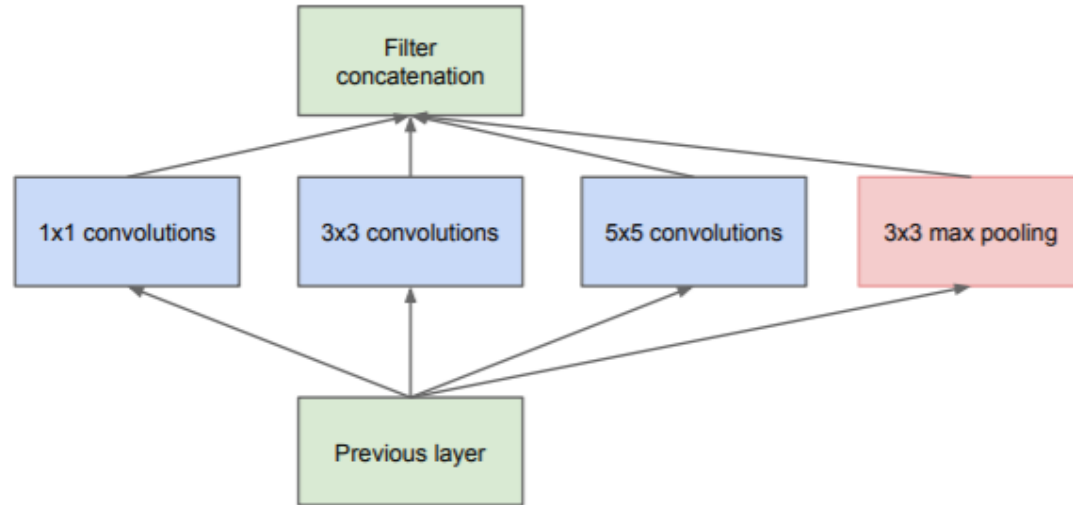
Inception module

GoogLeNet (Szegedy et al., 2014)

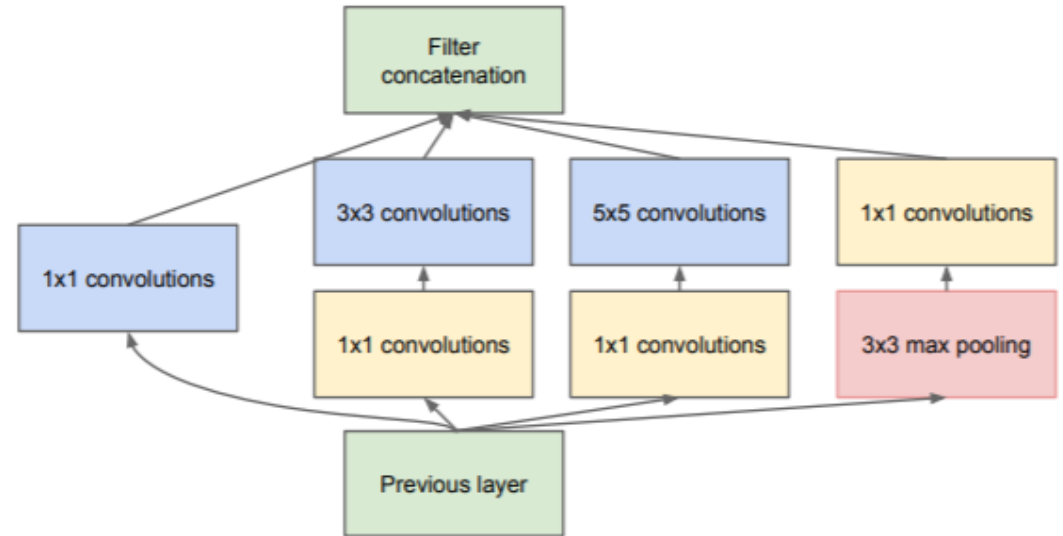


(a) Inception module, naïve version

GoogLeNet (Szegedy et al., 2014)

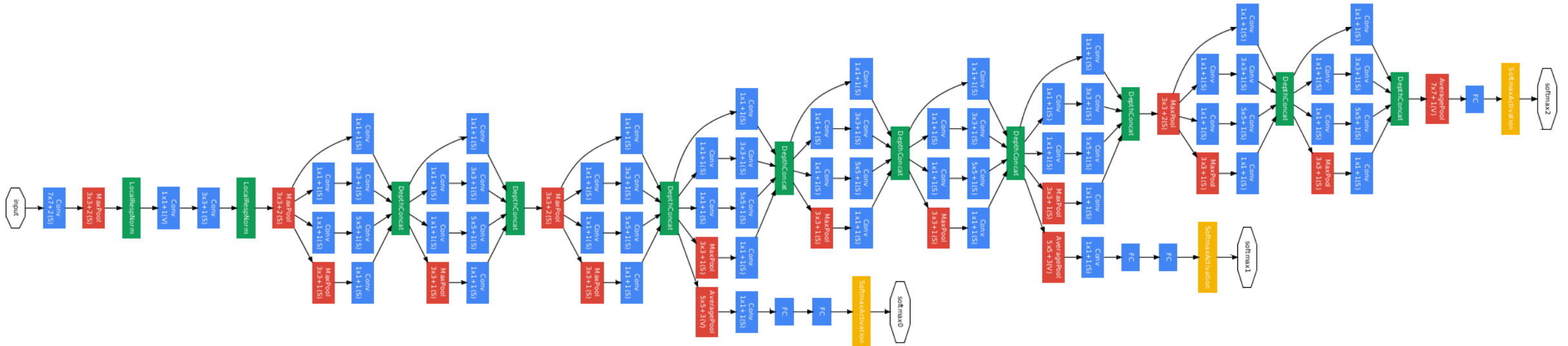


(a) Inception module, naïve version



(b) Inception module with dimension reductions

GoogLeNet (Szegedy et al., 2014)



ResNet (He et al., 2015)

Deep Residual Learning for Image Recognition

Kaiming He Xiangyu Zhang Shaoqing Ren Jian Sun

Microsoft Research

{kahe, v-xiangz, v-shren, jiansun}@microsoft.com

ResNet (He et al., 2015)

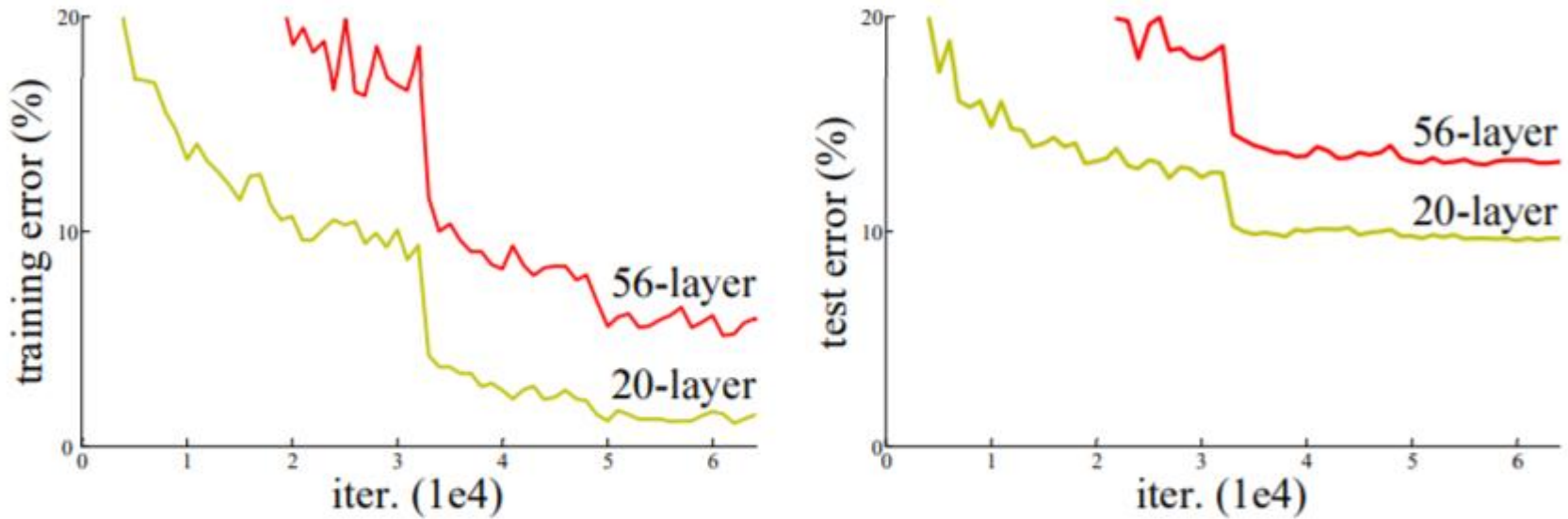


Figure 1. Training error (left) and test error (right) on CIFAR-10 with 20-layer and 56-layer “plain” networks. The deeper network has higher training error, and thus test error. Similar phenomena on ImageNet is presented in Fig. 4.

Residual Block

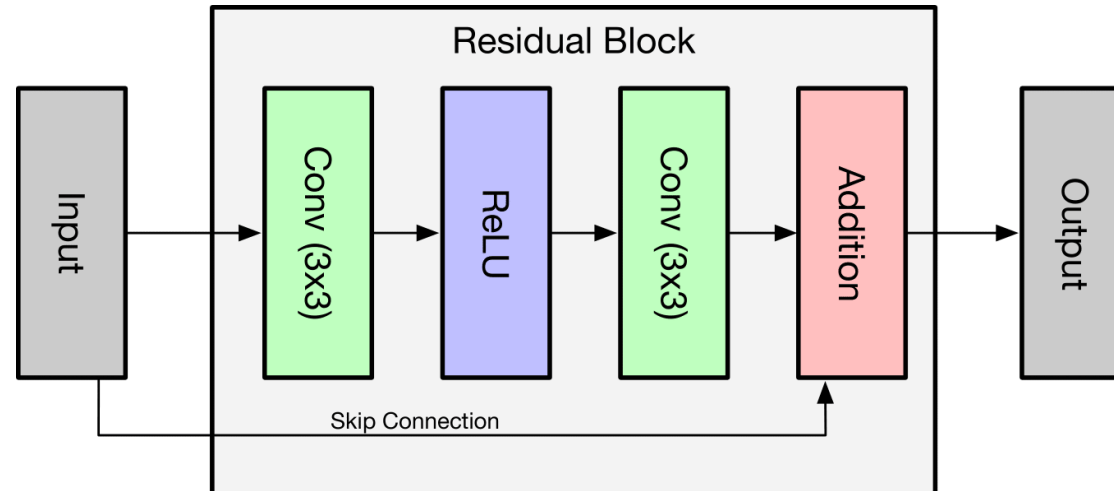
$F(x)$ is a sequence of regular layers

In a residual block we merge $F(x)$ with the input x and return

$$H(x) = F(x) + x$$

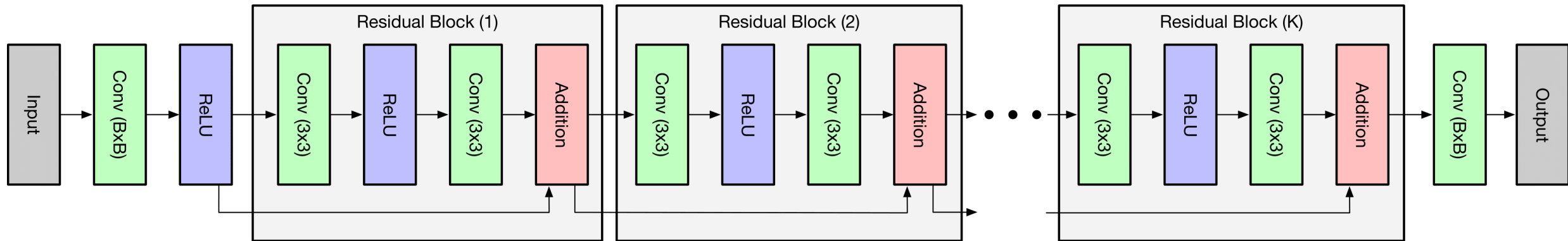
The addition of x to the output of the computation is called “skip connection”.

A residual block can contain any number of layers and any type



ResNet (He et al., 2015)

ResNet is composed of a sequence of residual blocks, in addition to standard layers:



Allows training very deep networks

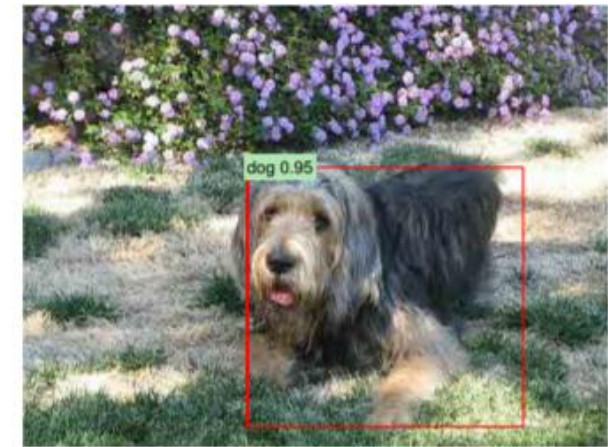
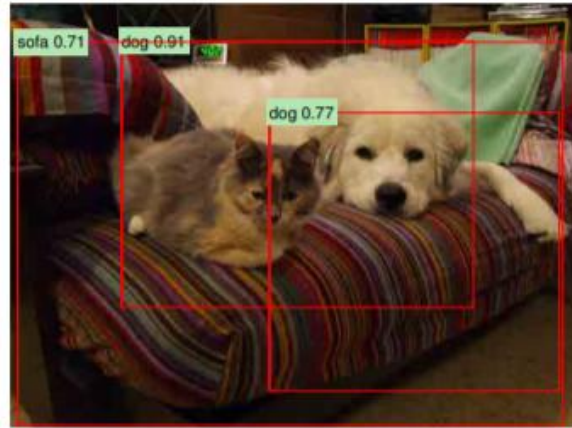
- Vanishing gradient
- Shallow to deep architecture

152 layers

Batch normalization after each conv layer

Other Computer Vision Tasks

Classification
Detection



Images from “Rich feature hierarchies for accurate object detection and semantic segmentation” (Girshick et al. 2014)

Other Computer Vision Tasks

Classification

Detection

Semantic segmentation



Images from “Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation” (Chen et al. 2018)

Next week:

NN Part 2